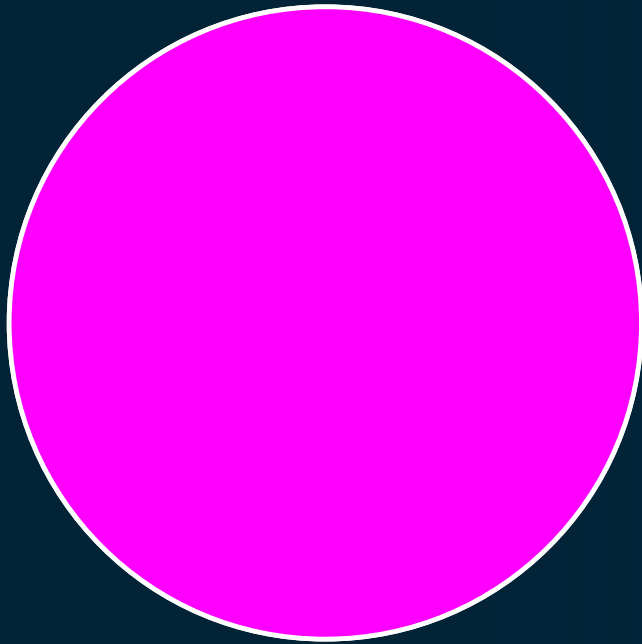


Token Limits, Rate Controls & Modality Restrictions

Chapter 08 · Episode 01 · Domain 2.3 — Resource Governance as Defense



Learning Objectives

1. Explain token ceilings, rate limits & modality restriction as one defense layer
2. Map volume attacks — DoS, cost amplification, model extraction — to the missing control
3. Recognize monitoring integration as what turns a limit into a defense

Three Volume-Based Attacks



EXHAUSTION

Compute drained by volume
Degraded AI delays detection



AMPLIFICATION

Oversized context submissions
Economic, not informational



OPERATIONAL DoS

Unlimited request bursts
Overwhelms infrastructure

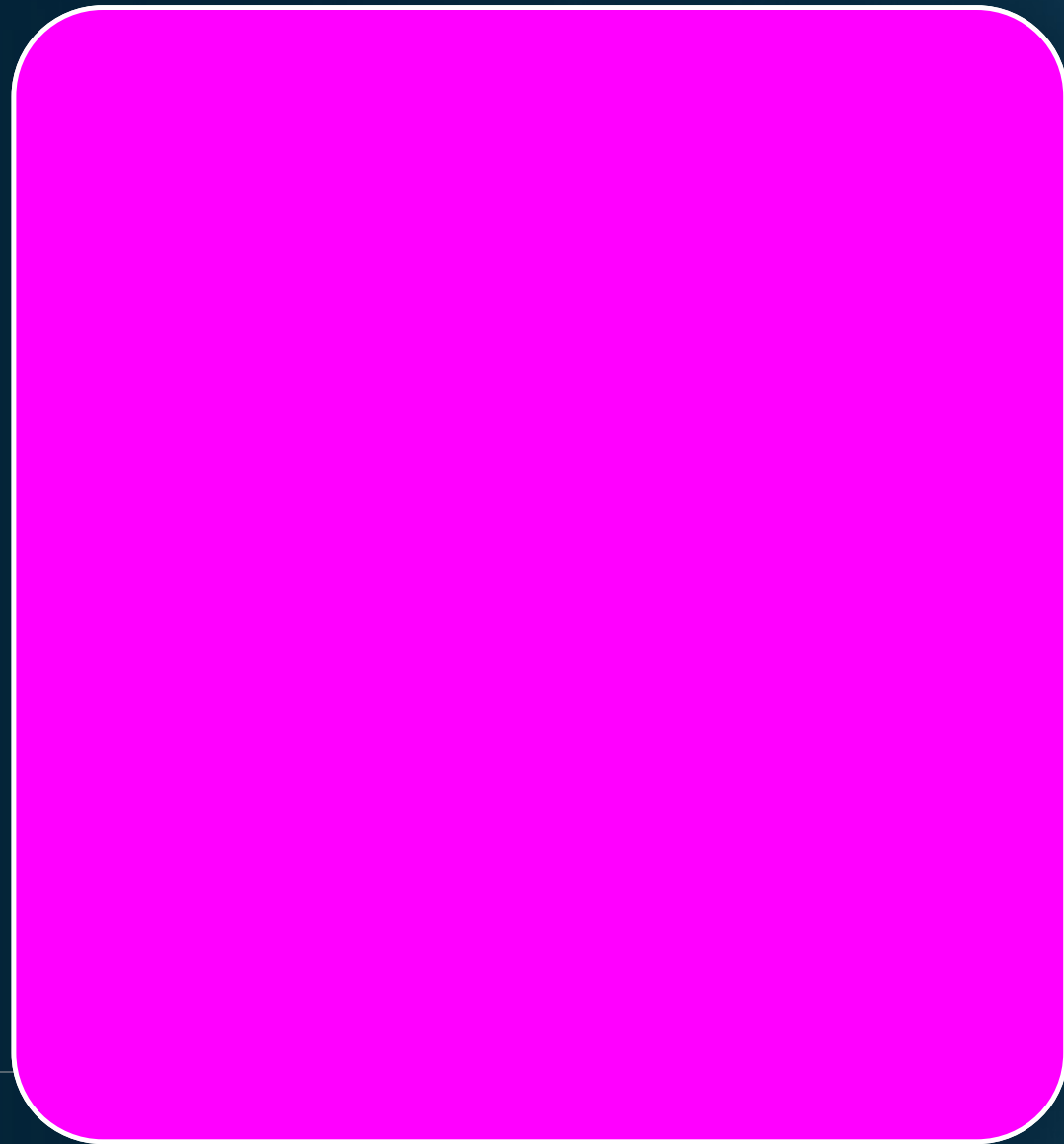
Set Maximum Token Ceilings

Caps context per inference cycle

Bounds cost AND manipulation surface

Unbounded windows expose earlier data

Limiting retention → data minimization



Rate Limiting

MECHANISMS

Per-user request thresholds

Time-based throttling

Burst detection

Daily/session quotas

WHY IT MATTERS

Primary defense vs model extraction

Reconstruction needs high query volume

Preventative + detective

Weak limits → scraping & recon

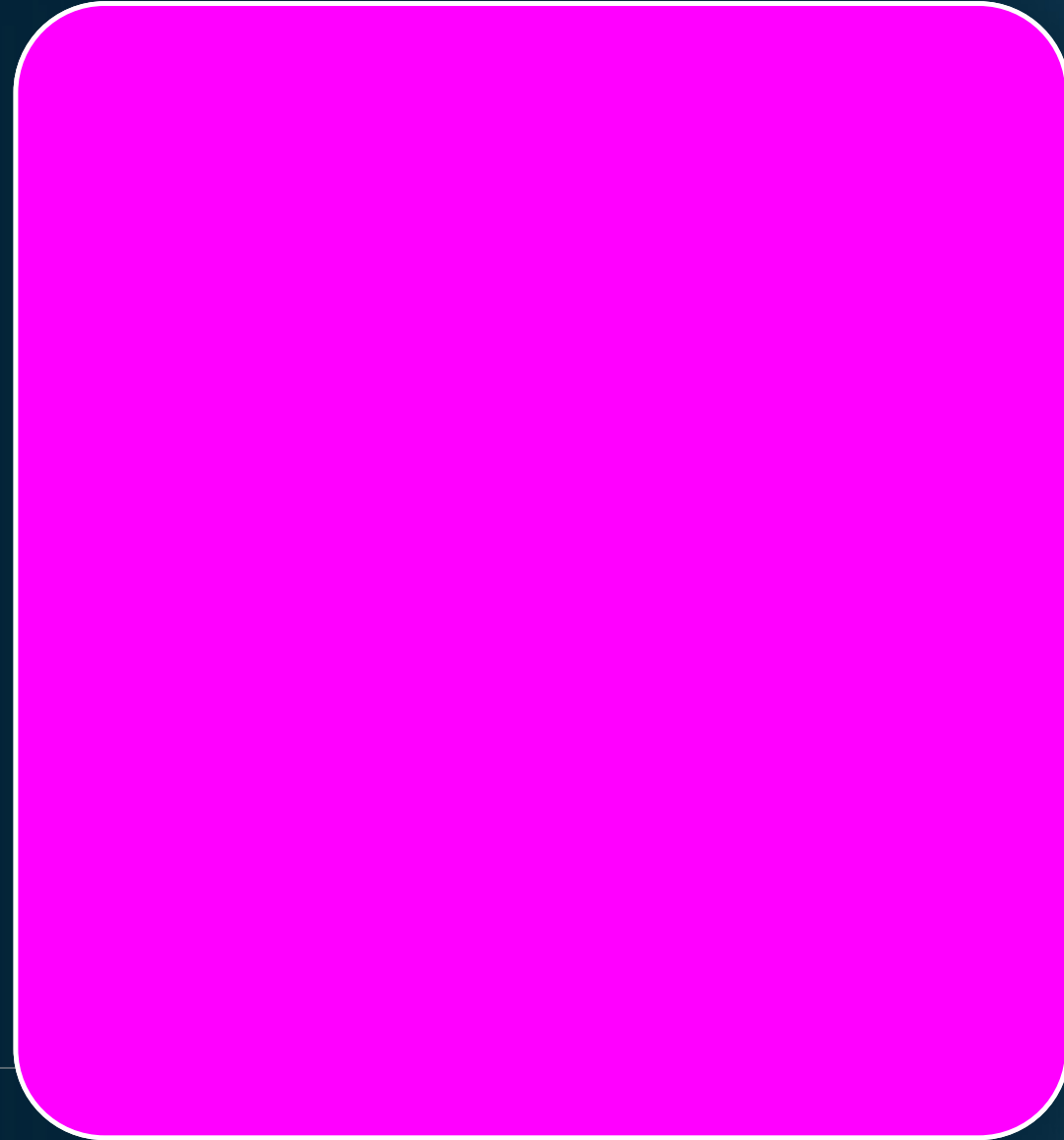
Modality Restriction

More modalities = more attack surface

Capability minimization at the gateway

Validate file format & size before parsing

Parsers/encoders carry their own bugs



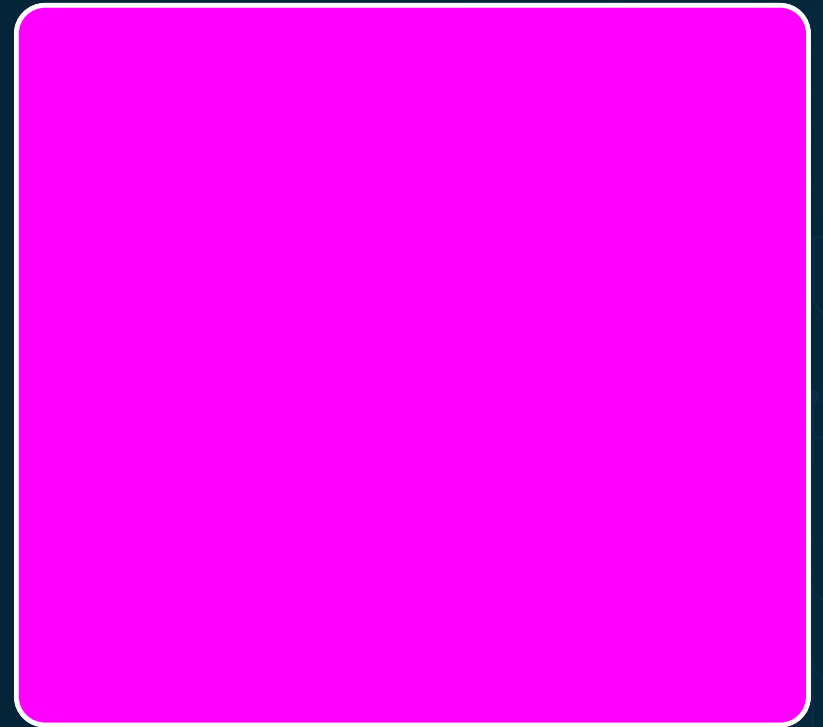
Controls Need Monitoring to Count

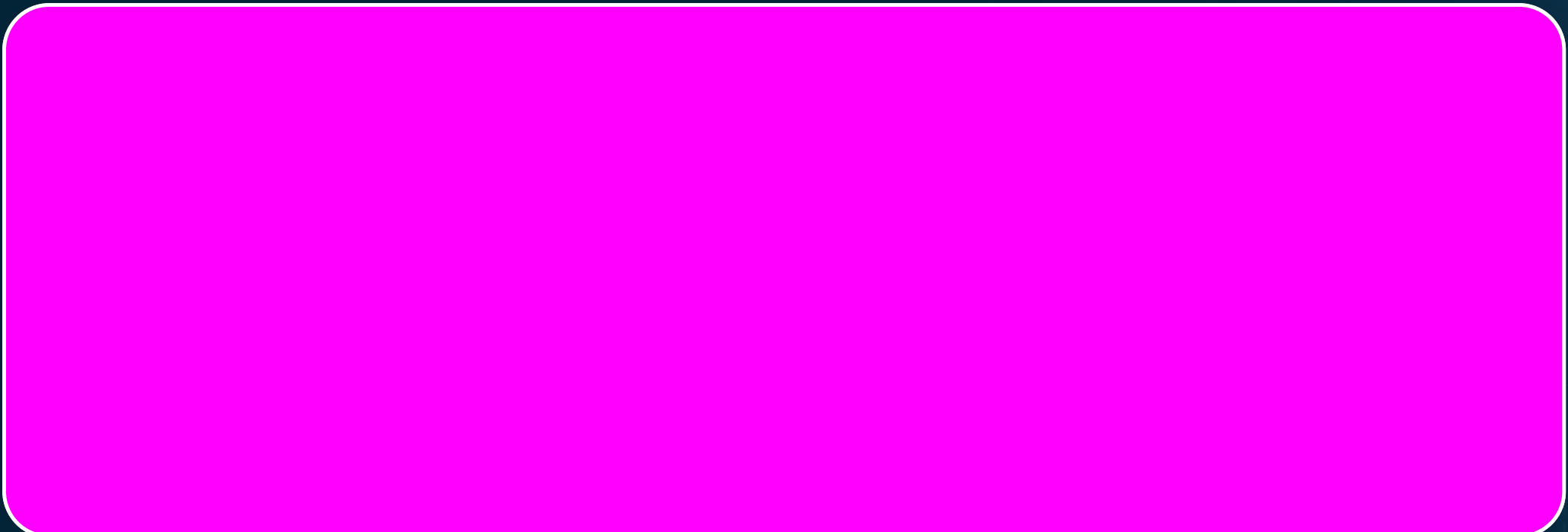
Usage telemetry = the baseline

Cost anomaly detection flags abuse

Adaptive rate tightens in real time

A limit that never alerts = blind spot





Demo: Resource Governance Layer

Token-ceiling enforcement, rate limiting with burst detection, modality restriction, cost telemetry & adaptive alerts

Exam Tips



RESOURCE = SECURITY

Not just performance



MODALITY

Capability min at the gateway



RATE LIMITS

Primary anti-extraction defense

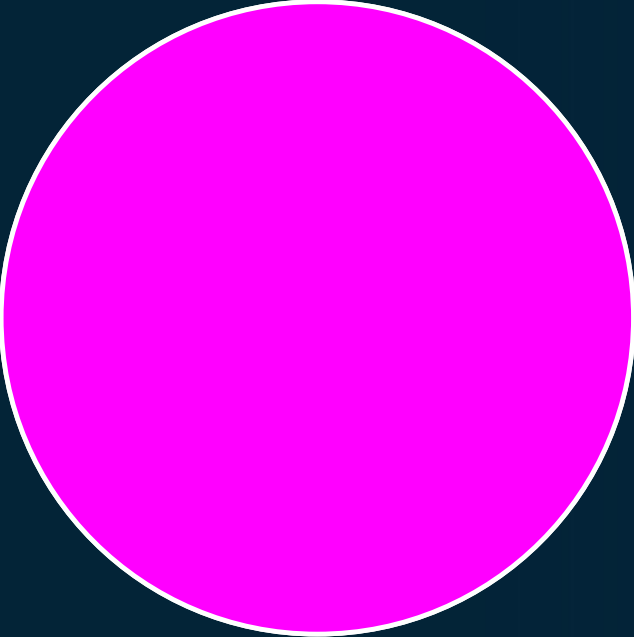


COST SPIKES

Detective control, not billing

Endpoint Protection & API Security

Chapter 08 · Episode 02 · Domain 2.3 — Inference APIs as Critical Infrastructure



Learning Objectives

1. Treat inference APIs as critical infrastructure, not auxiliary services
2. Pair authentication and authorization — both required, neither sufficient alone
3. Enforce TLS everywhere and wire endpoint telemetry into the SIEM

Authentication: Prove the Caller

MECHANISMS

API keys = basic ID, weak alone
Token-based: expiry, scope, revoke
OAuth / federated identity
Service-to-service auth internally

EXAM CONTENT

Eliminate anonymous access in prod
Even read-only enables extraction
Auth precedes every inference call
Inherit from enterprise IAM

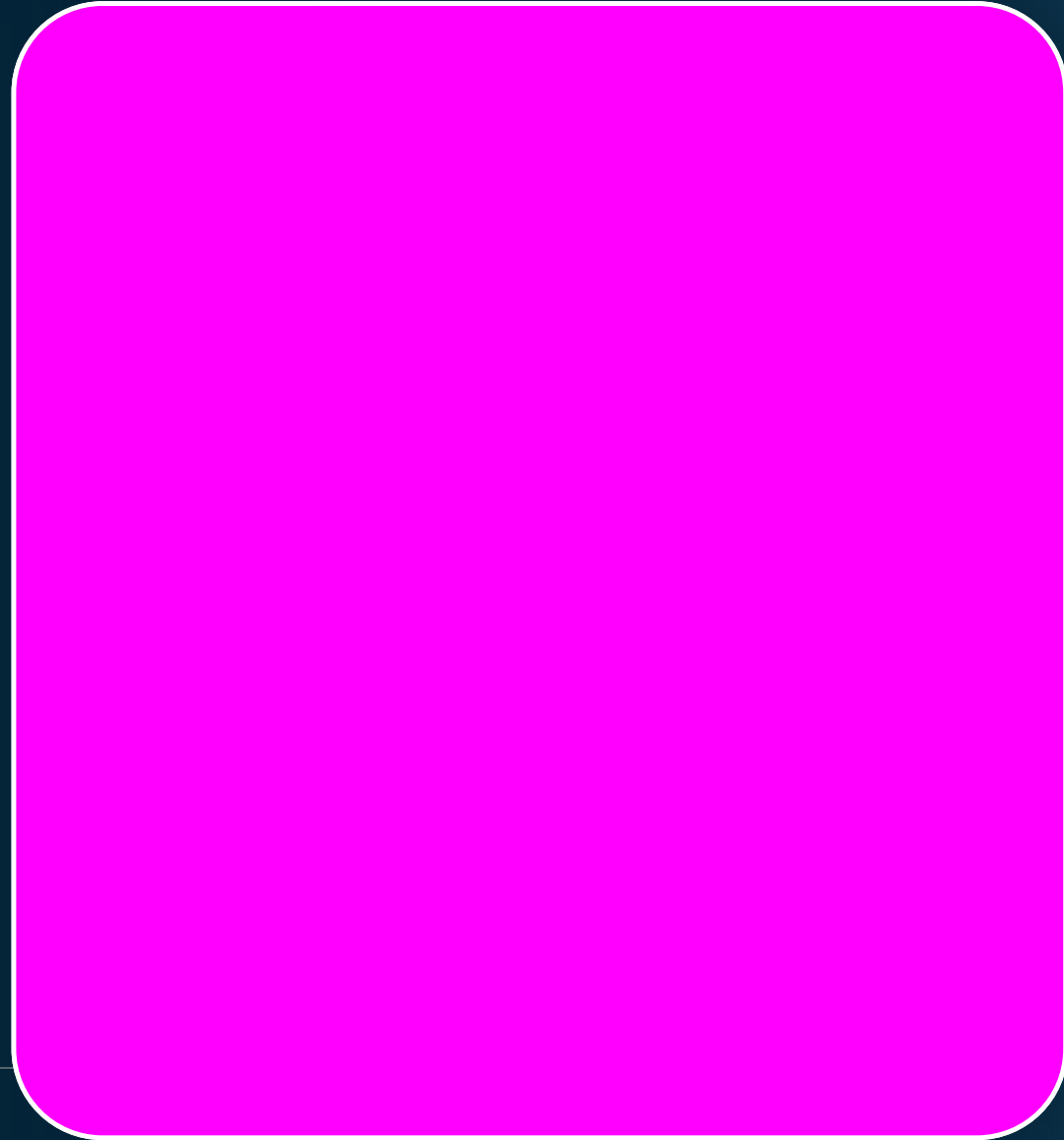
Authorization: Scope What They Do

RBAC by role: analyst, admin, developer

ABAC adds context: dept, risk, geo

Scope inference / fine-tune / config /
retrieval separately

Inference priv modifying training = least-
privilege break



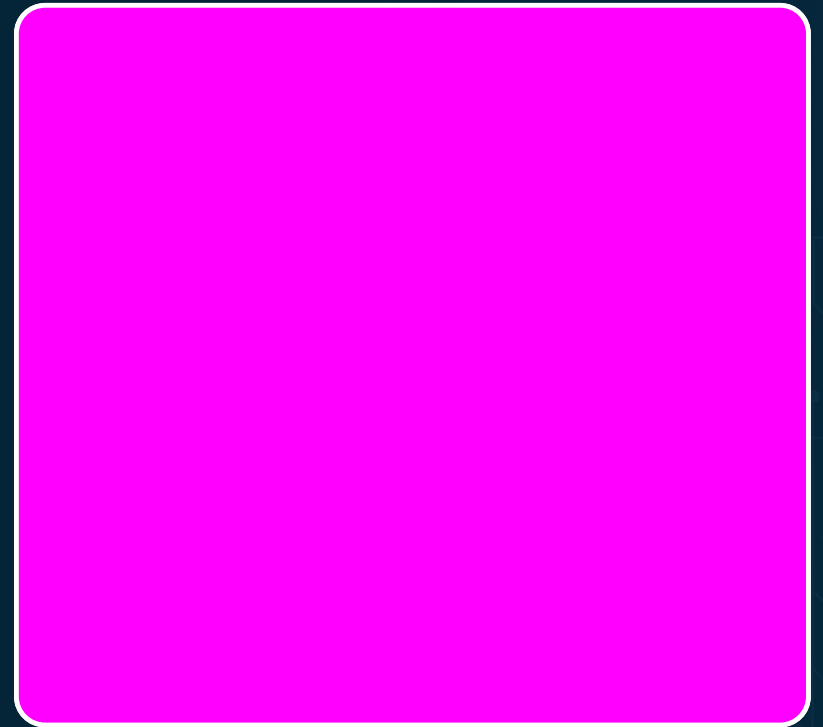
Encrypt Every Channel — TLS

TLS protects prompts & responses in transit

Certificate validation stops impersonation

Internal traffic needs TLS too

Trust-the-internal-network is a blind spot



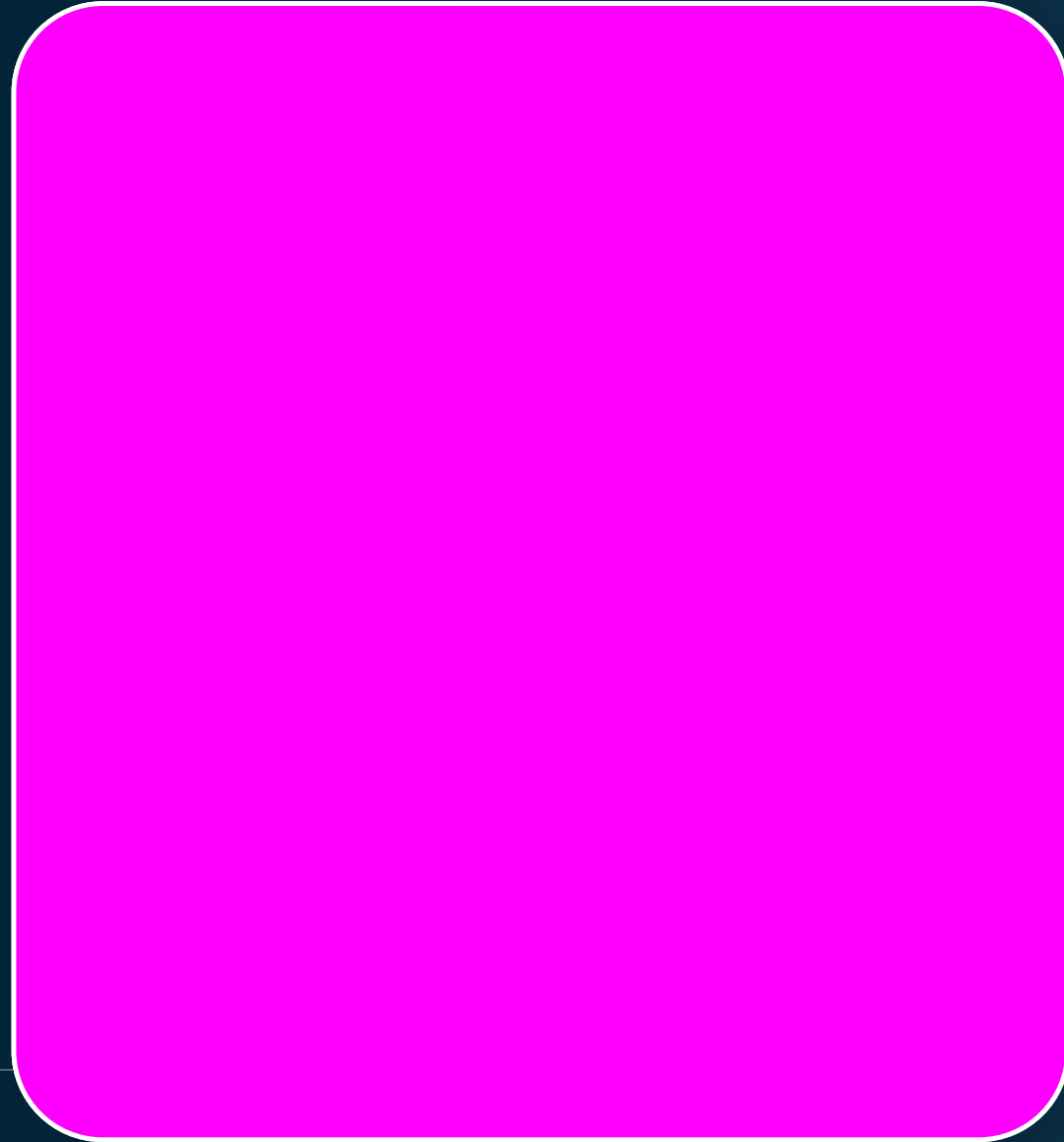
Endpoint Monitoring = Visibility

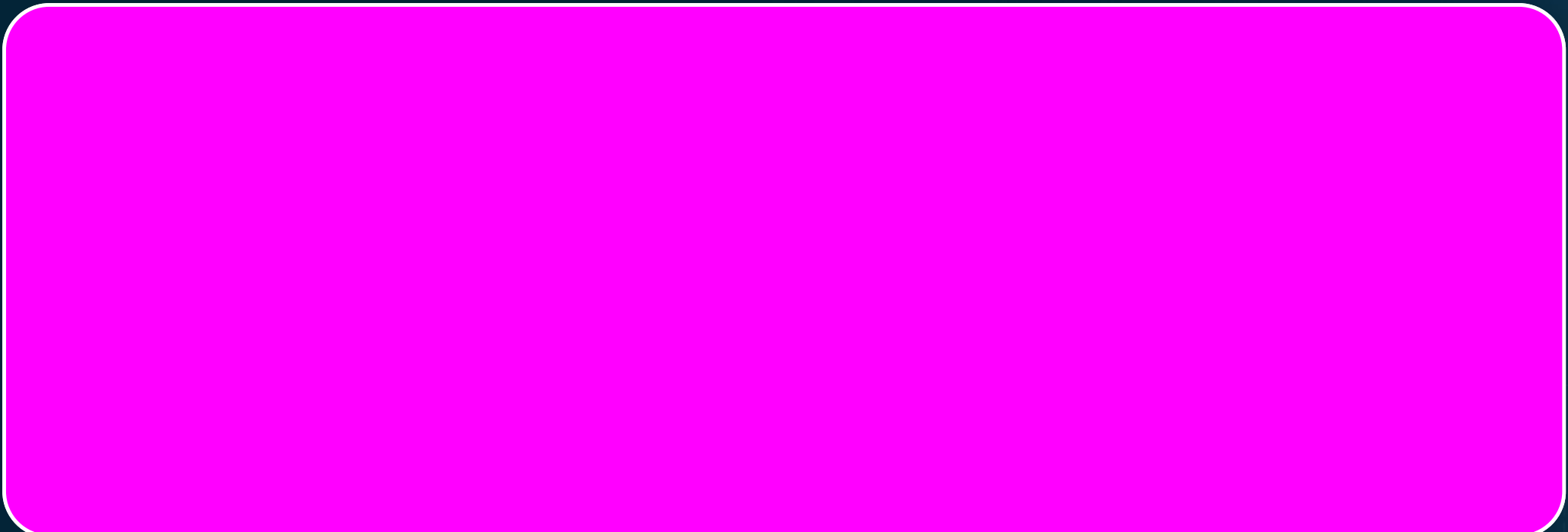
Telemetry: request rate, tokens,
errors, params

Detect bursts, structured probing,
odd params

Correlate with identity → catch stolen creds

Alerts must trigger revoke + isolate,
feed SIEM





Demo: AI API Security Audit

Auth strength, authorization scoping, TLS enforcement, enumeration resistance
& monitoring integration checks

Exam Tips



CRITICAL INFRA
Treat like
financial/identity APIs



TLS INTERNAL
Internal networks aren't trusted



AUTHN + AUTHZ
Both required, different

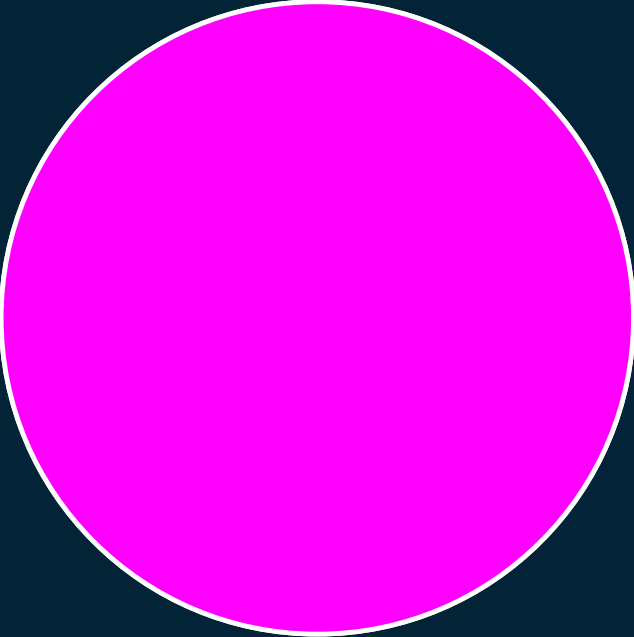


SIEM
Standalone logs insufficient



Endpoint Protection & API Security

Chapter 08 · Episode 02 · Domain 2.3 — Inference APIs as Critical Infrastructure



Learning Objectives

1. Treat inference APIs as critical infrastructure, not auxiliary services
2. Pair authentication and authorization — both required, neither sufficient alone
3. Enforce TLS everywhere and wire endpoint telemetry into the SIEM

Authentication: Prove the Caller

MECHANISMS

API keys = basic ID, weak alone
Token-based: expiry, scope, revoke
OAuth / federated identity
Service-to-service auth internally

EXAM CONTENT

Eliminate anonymous access in prod
Even read-only enables extraction
Auth precedes every inference call
Inherit from enterprise IAM

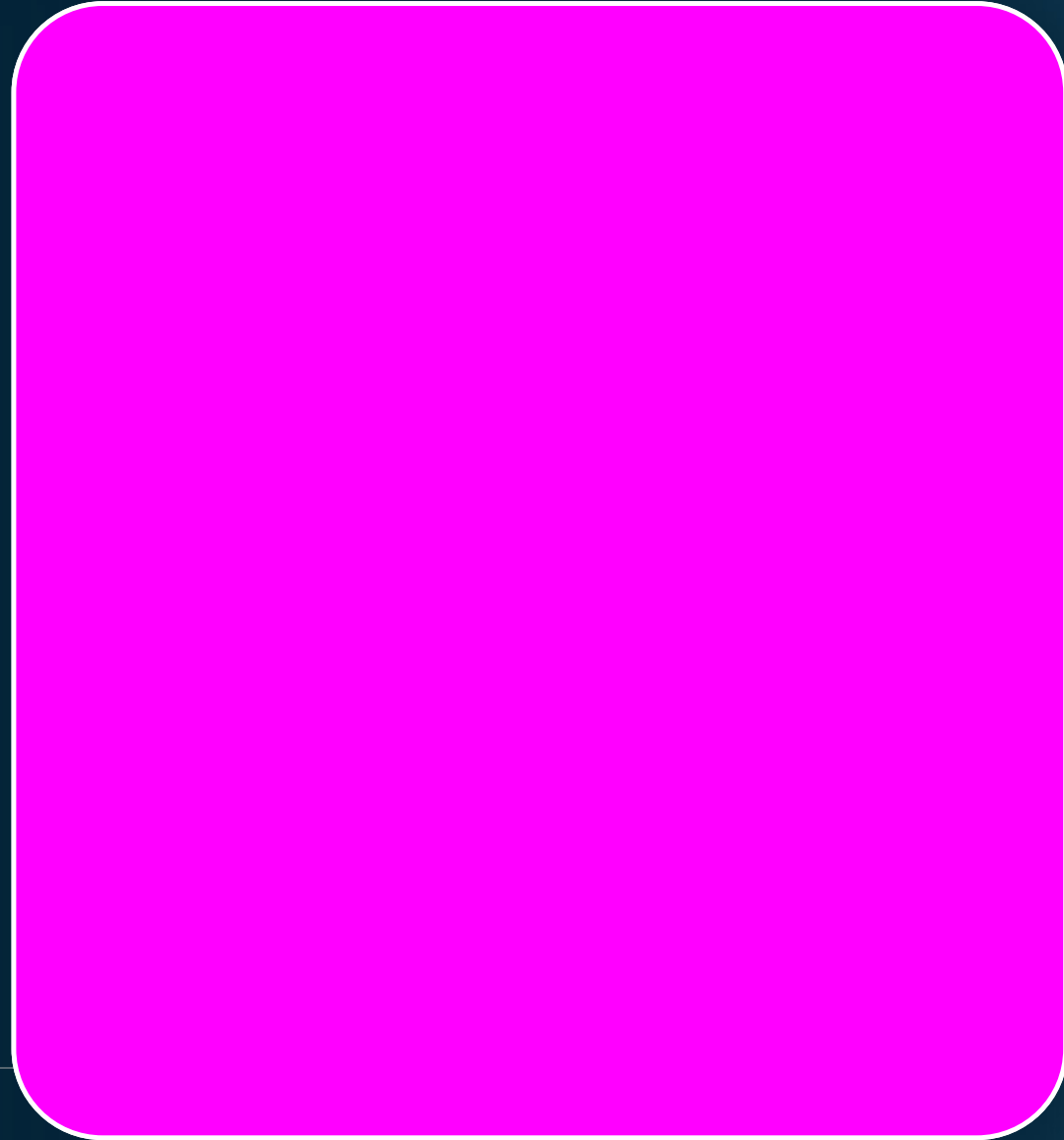
Authorization: Scope What They Do

RBAC by role: analyst, admin, developer

ABAC adds context: dept, risk, geo

Scope inference / fine-tune / config /
retrieval separately

Inference priv modifying training = least-
privilege break



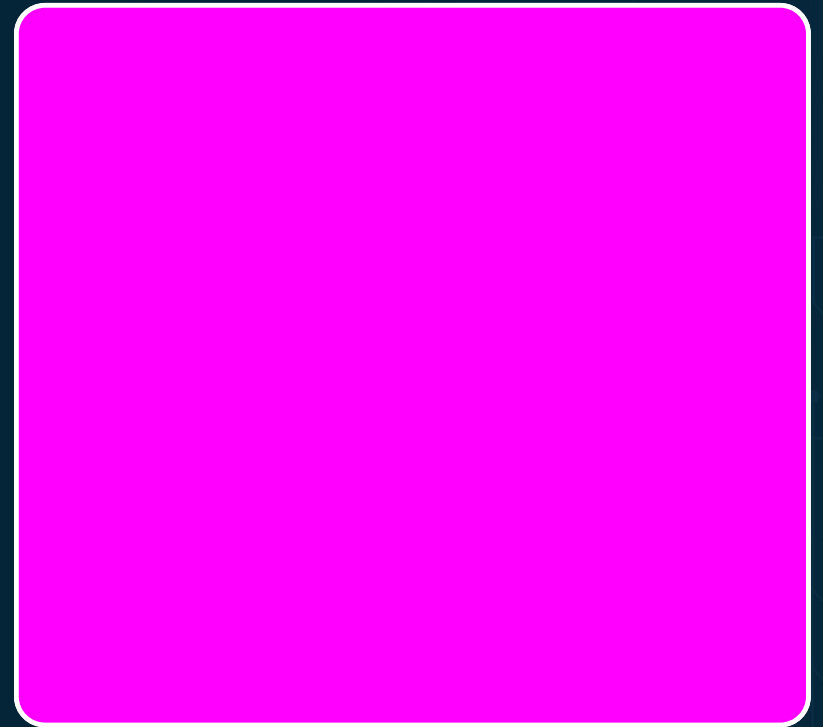
Encrypt Every Channel — TLS

TLS protects prompts & responses in transit

Certificate validation stops impersonation

Internal traffic needs TLS too

Trust-the-internal-network is a blind spot



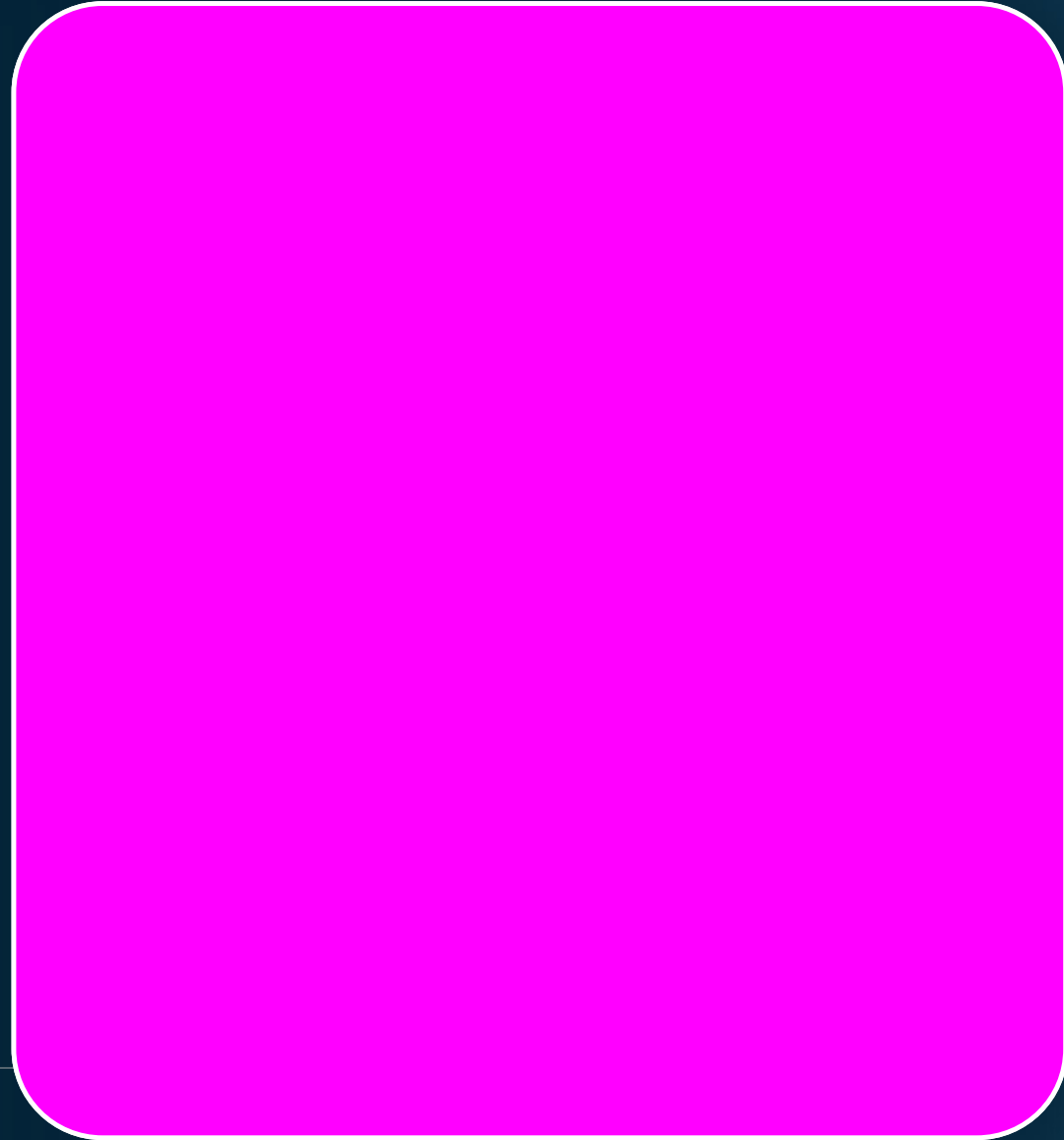
Endpoint Monitoring = Visibility

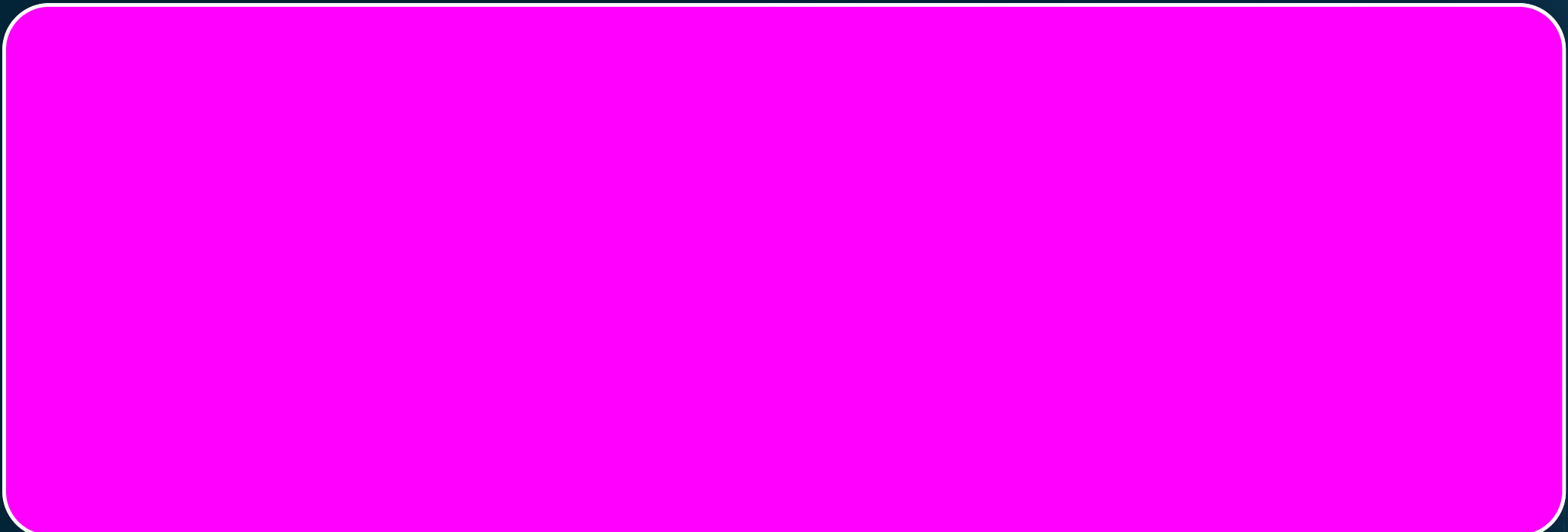
Telemetry: request rate, tokens,
errors, params

Detect bursts, structured probing,
odd params

Correlate with identity → catch stolen creds

Alerts must trigger revoke + isolate,
feed SIEM





Demo: AI API Security Audit

Auth strength, authorization scoping, TLS enforcement, enumeration resistance
& monitoring integration checks

Exam Tips



CRITICAL INFRA
Treat like
financial/identity APIs



TLS INTERNAL
Internal networks aren't trusted



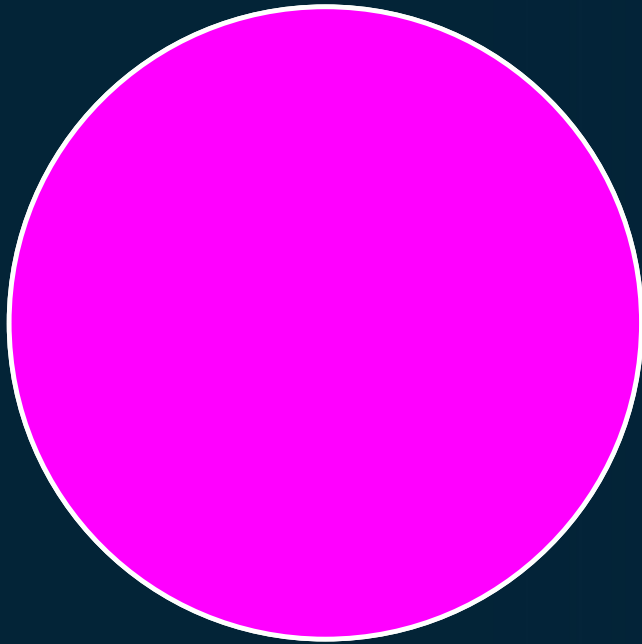
AUTHN + AUTHZ
Both required, different



SIEM
Standalone logs insufficient

Model Access Controls & Privilege Boundaries

Chapter 08 · Episode 03 · Domain 2.3 — Who Can Query, Modify, Retrain



Learning Objectives

1. Separate the three access boundaries: query, modify, retrain
2. Layer RBAC with ABAC scoping and environment boundaries
3. Protect model weights as IP and audit for privilege creep

RBAC and ABAC Scoping

RBAC — BY ROLE

Analyst → inference only

Engineer → retrain, hyperparams

Auditor → read-only logs

ABAC — BY CONTEXT

Dept, risk classification

Time of access, geo location

Retrain needs role + approved segment

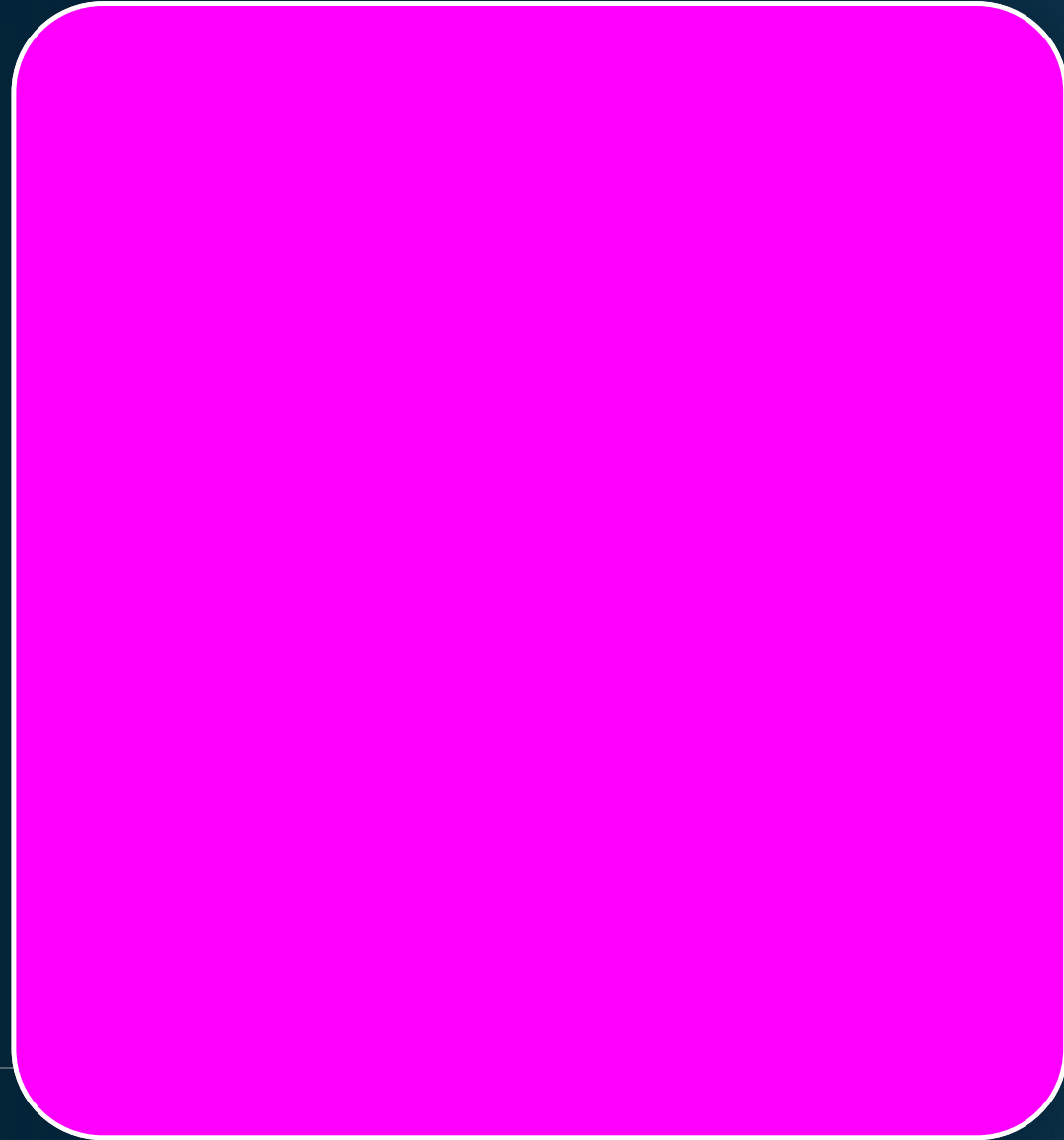
Segment Dev, Staging & Production

Access to one environment $\neq$ access to another

Isolate experimental variants from prod

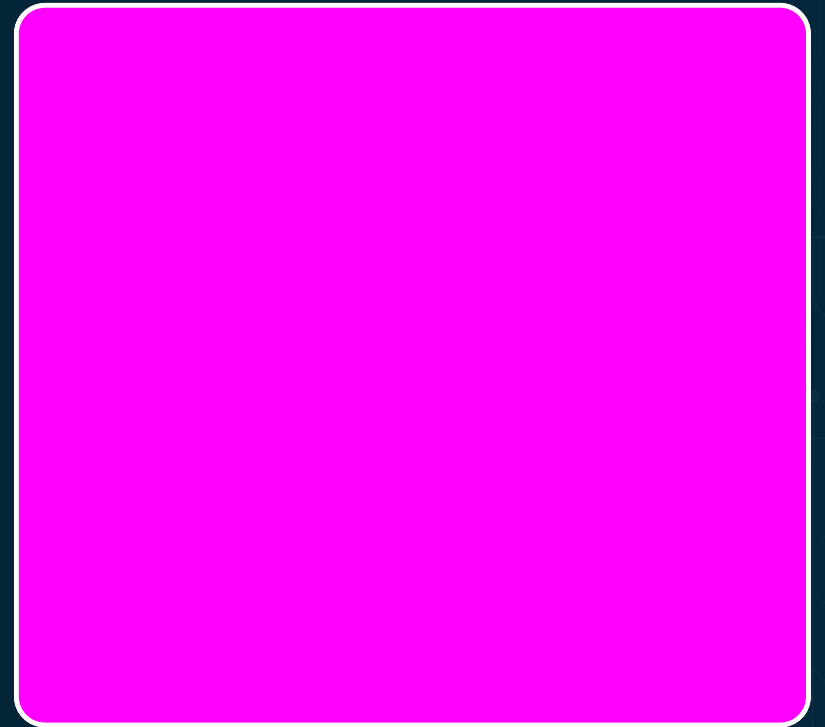
Deployment approval workflows
gate promotion

Config drift introduces boundary surprises



Model Weights Are IP Assets

Stolen weights → cloned models, exposed data
Tightly control export/download permissions
Encrypt artifacts at rest
Monitor abnormal downloads / large transfers



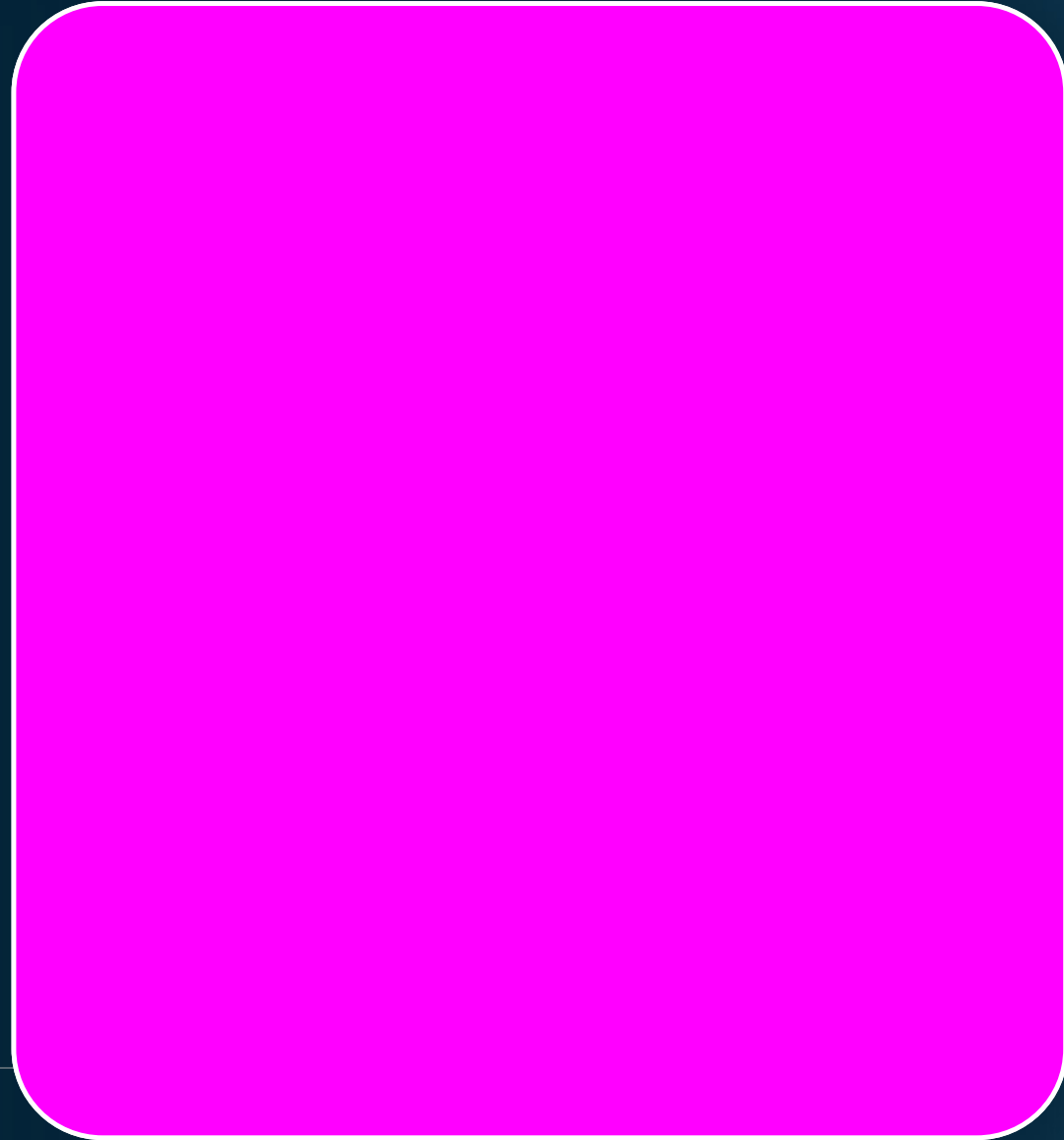
Audit for Privilege Creep

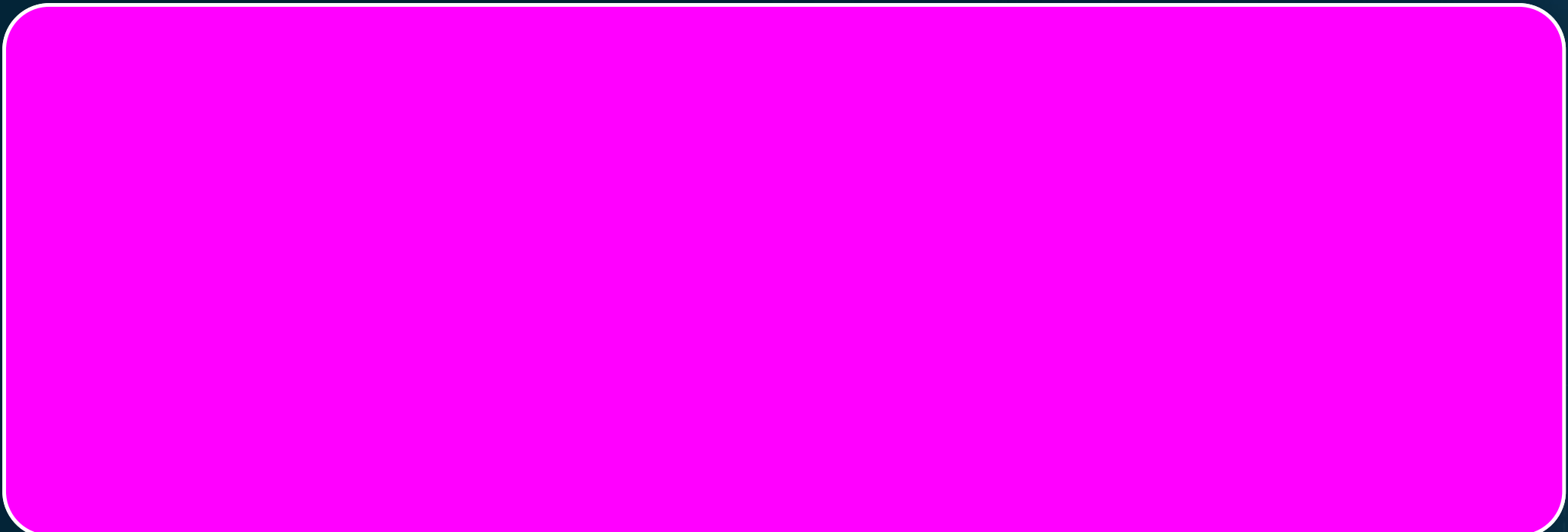
Periodic privilege audits =
only-what's-needed

Log retraining, config, deployment approvals

Alert on escalation, config change,
model swap

Configure-once-and-forget is the
failure mode





Demo: Model Access Control Audit

RBAC, ABAC scoping, environment segmentation, weight-artifact protection
& privilege-creep detection via log review

Exam Tips



3 BOUNDARIES

Query / modify / retrain



ENVIRONMENTS

Dev compromise $\neq$ prod reach



RBAC vs ABAC

Role vs contextual attributes

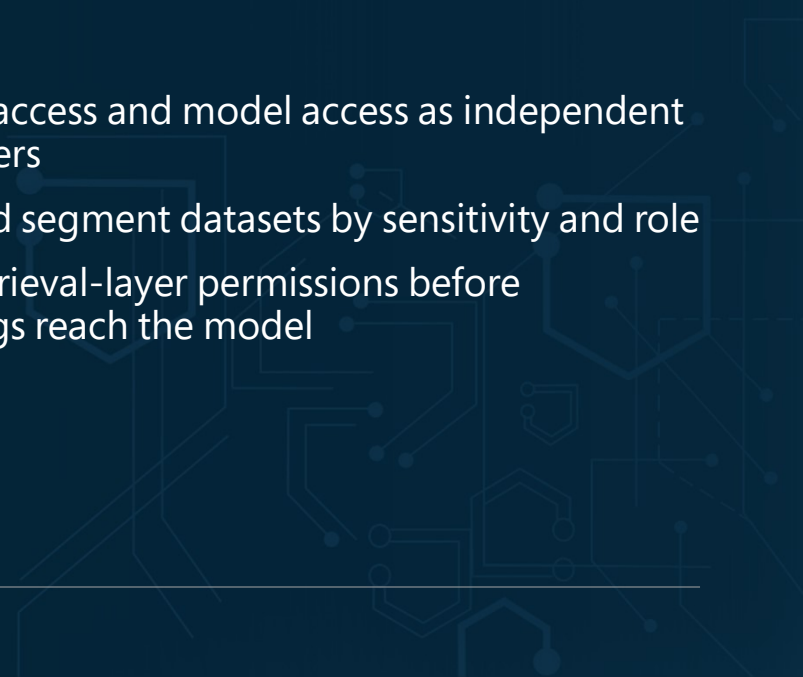
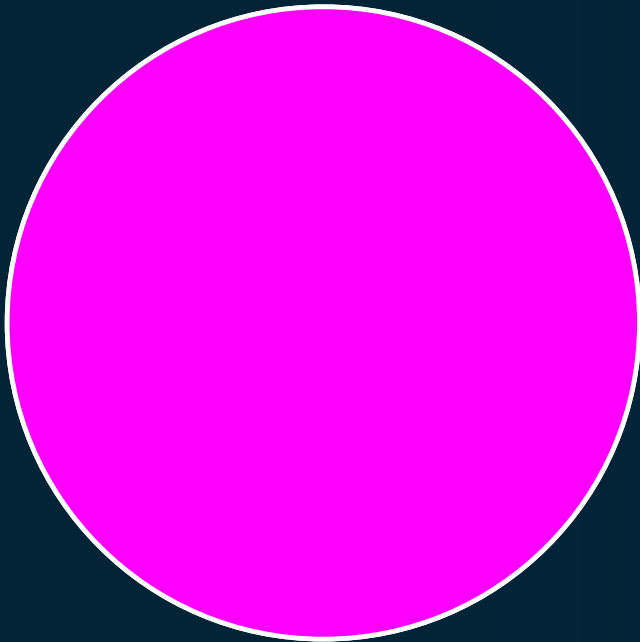


WEIGHTS = IP

Export + encrypt + monitor

Data Access Controls in AI Systems

Chapter 08 · Episode 04 · Domain 2.3 — Data Boundaries Are the Perimeter



Learning Objectives

1. Treat data access and model access as independent control layers
2. Classify and segment datasets by sensitivity and role
3. Enforce retrieval-layer permissions before embeddings reach the model

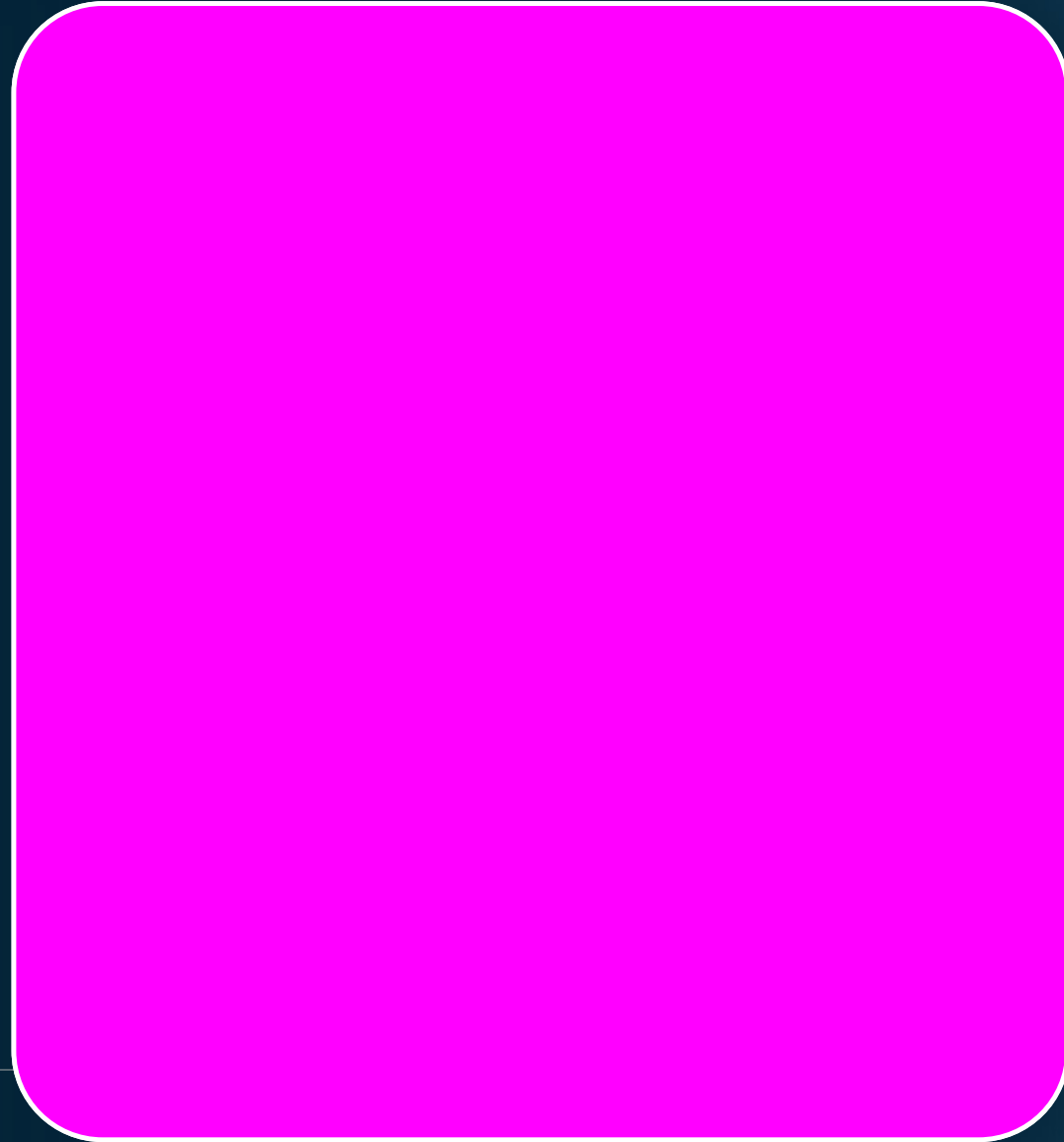
Segment Datasets by Sensitivity

Classify: public → internal →
confidential → regulated

Fraud DB → financial security team only

Endpoint telemetry → SOC analysts only

Watch cross-dataset exposure &
bias amplification



Read vs Write Privilege Separation

READ — MANY

Visibility for evaluation

Monitor excessive inspection

Inspection spikes = recon

WRITE — FEW

Alters dataset contents

Unauthorized write = poisoning

Validated ingestion pipelines only

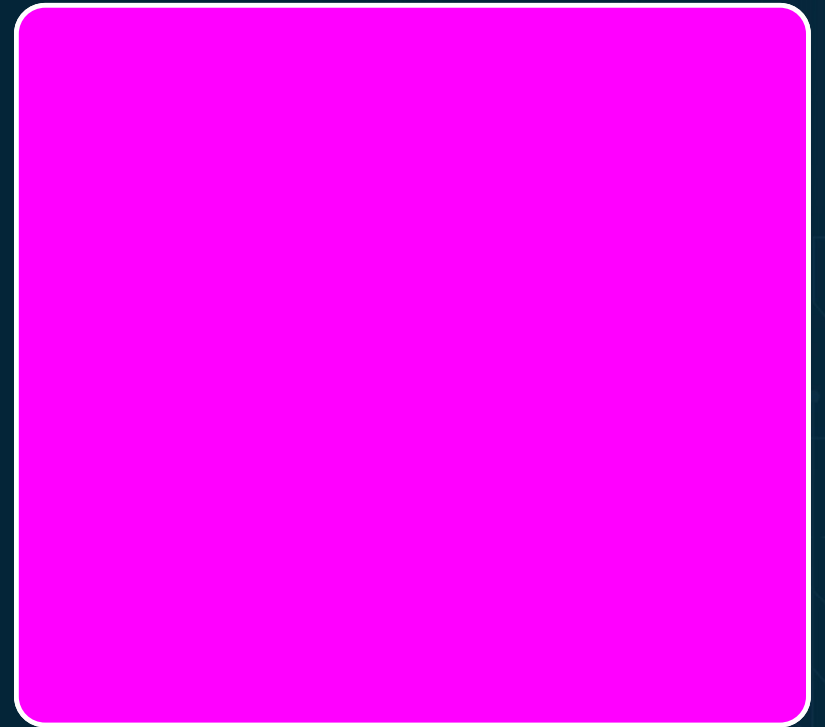
Retrieval Is Its Own Exposure Surface

Role-based retrieval: docs at/below clearance

Filter enforces BEFORE results reach model

Segment sensitive corpora from general KB

Inference-time overreach = retrieval governance fail



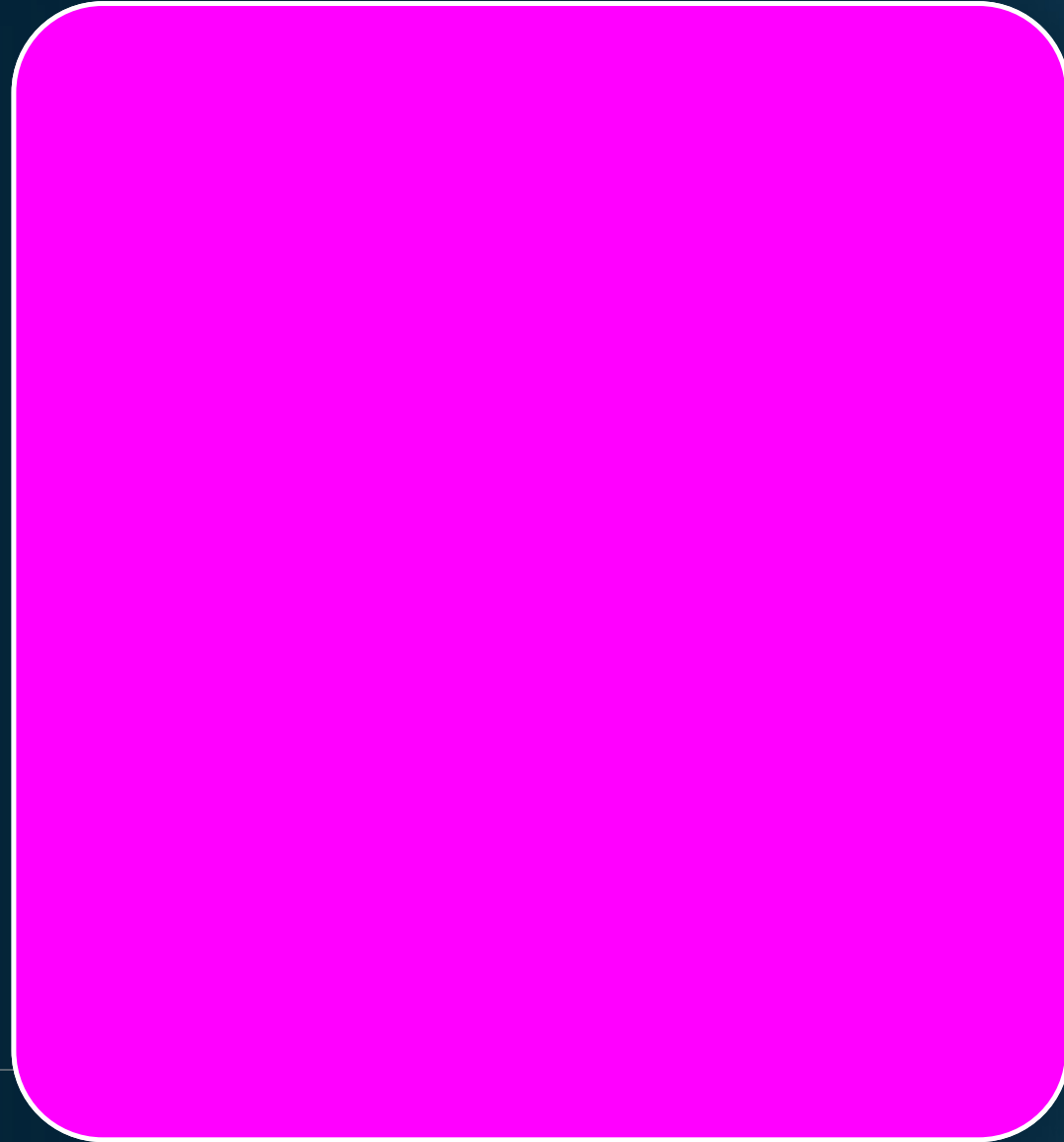
Audit the Data Layer

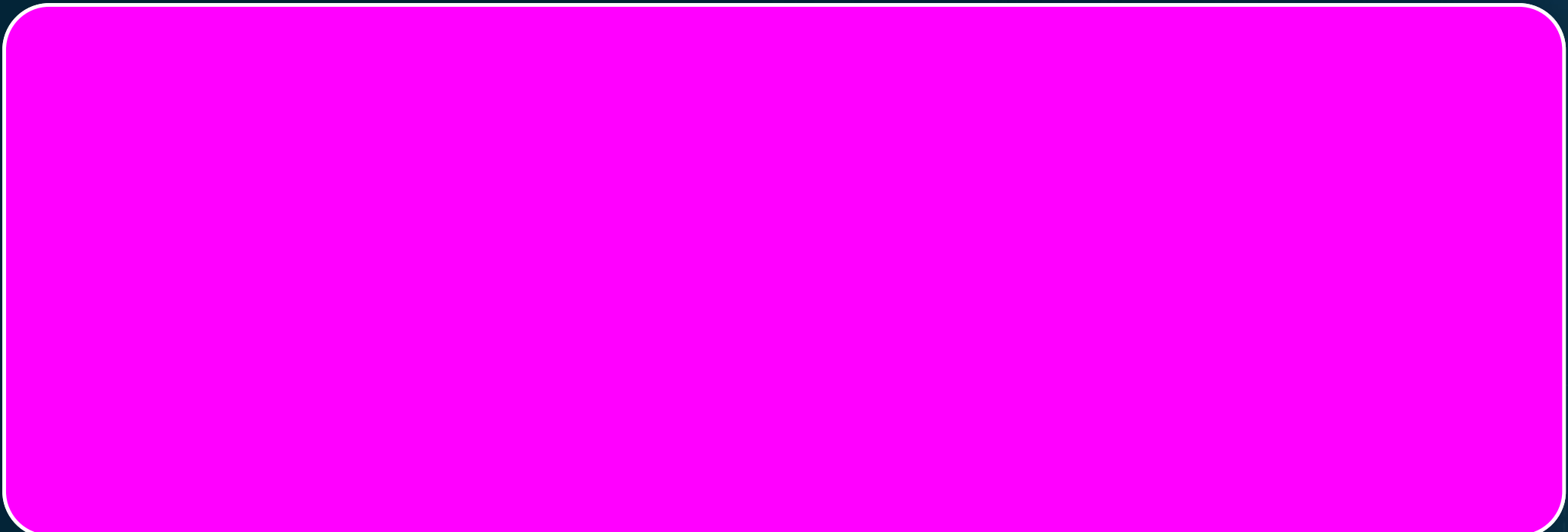
Log who accessed which datasets,
when, what role

Monitor unusual retrieval patterns
(indirect probing)

Correlate with identity: escalation,
geo anomalies

Review permissions each cycle —
stop privilege creep





Demo: Data Access Governance Audit

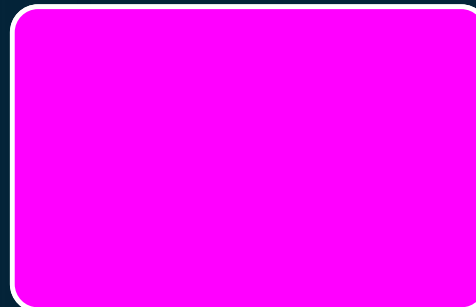
Dataset classification, RBAC, read/write separation, retrieval-layer permissions & access logging with anomaly detection

Exam Tips



TWO LAYERS

Data $\neq$ model — test both



READ vs WRITE

Least privilege on data



RAG SURFACE

Vector leakage is separate

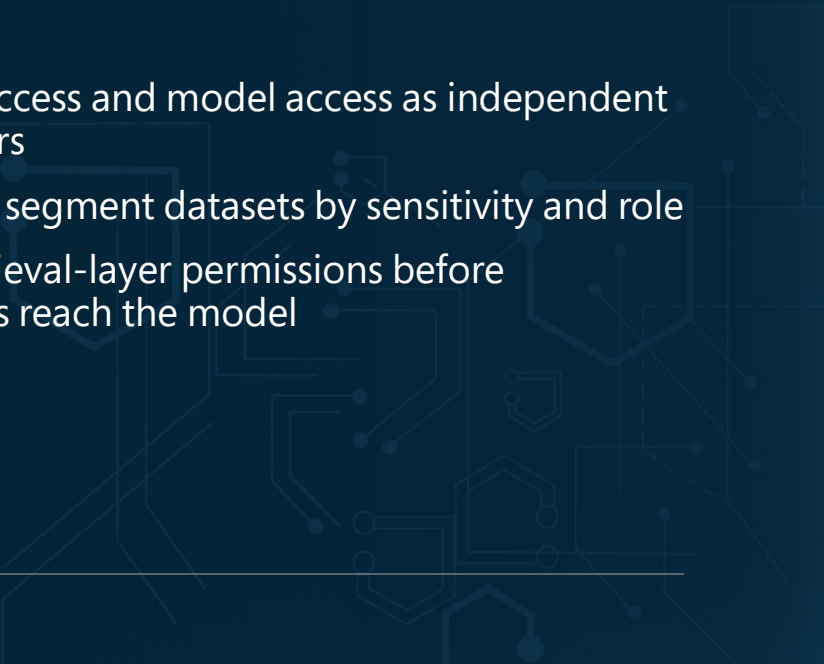
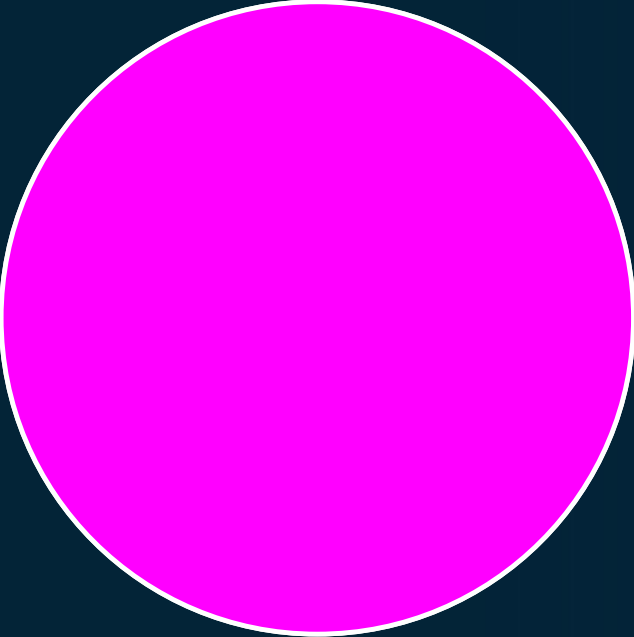


OVERREACH

Retrieval fail, not model fail

Data Access Controls in AI Systems

Chapter 08 · Episode 04 · Domain 2.3 — Data Boundaries Are the Perimeter



Learning Objectives

1. Treat data access and model access as independent control layers
2. Classify and segment datasets by sensitivity and role
3. Enforce retrieval-layer permissions before embeddings reach the model

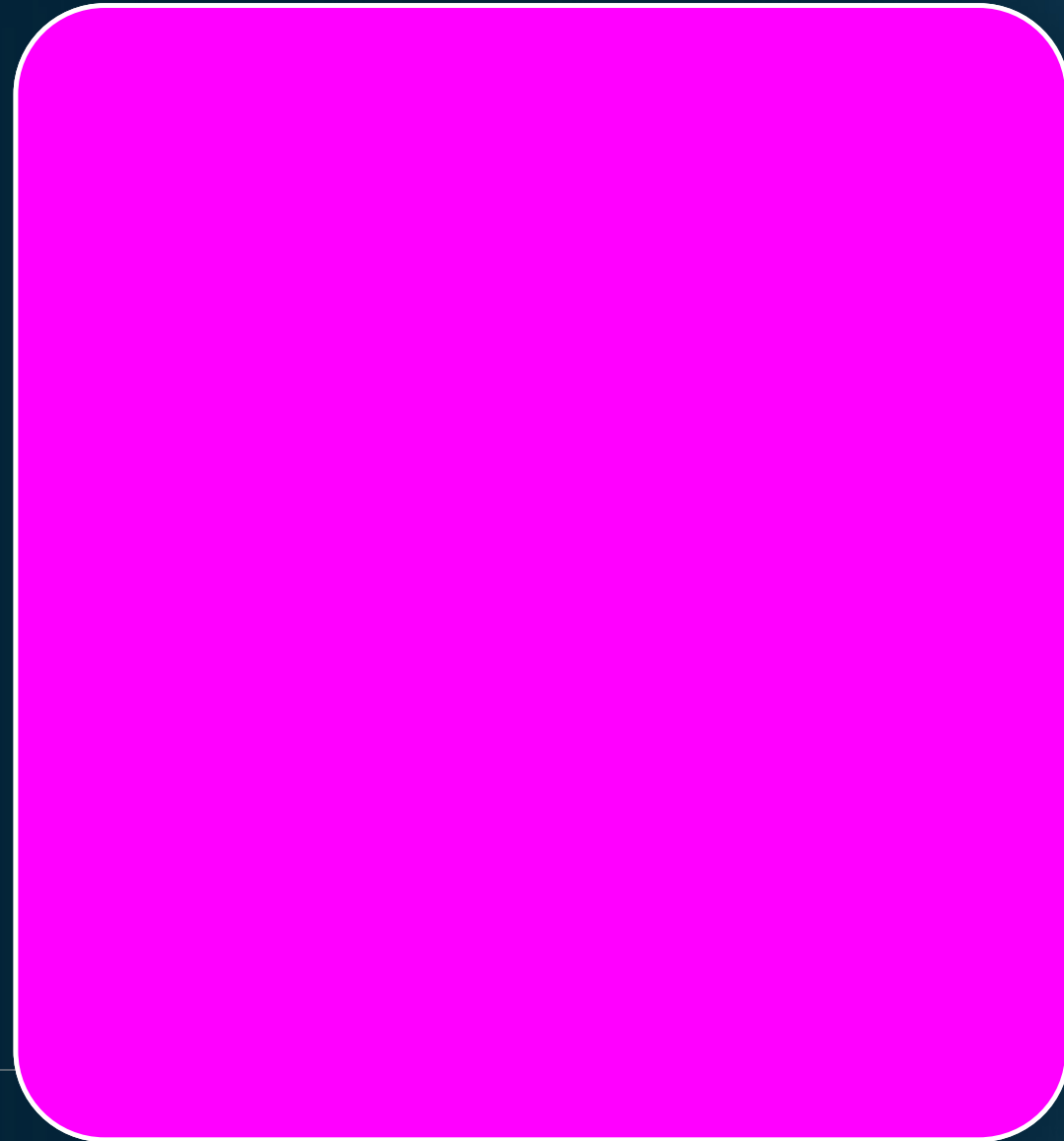
Segment Datasets by Sensitivity

Classify: public → internal →
confidential → regulated

Fraud DB → financial security team only

Endpoint telemetry → SOC analysts only

Watch cross-dataset exposure &
bias amplification



Read vs Write Privilege Separation

READ — MANY

Visibility for evaluation

Monitor excessive inspection

Inspection spikes = recon

WRITE — FEW

Alters dataset contents

Unauthorized write = poisoning

Validated ingestion pipelines only

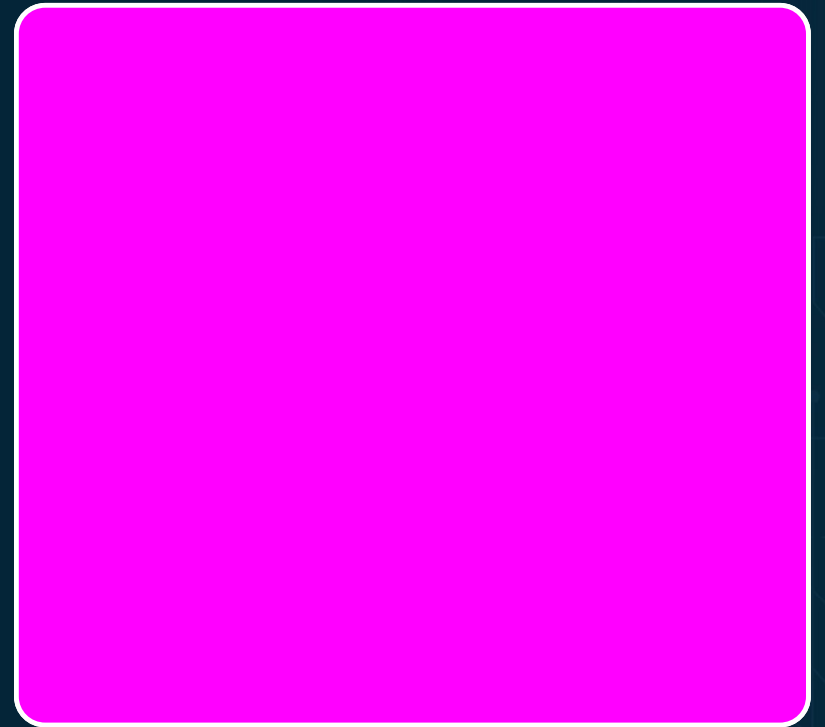
Retrieval Is Its Own Exposure Surface

Role-based retrieval: docs at/below clearance

Filter enforces BEFORE results reach model

Segment sensitive corpora from general KB

Inference-time overreach = retrieval governance fail



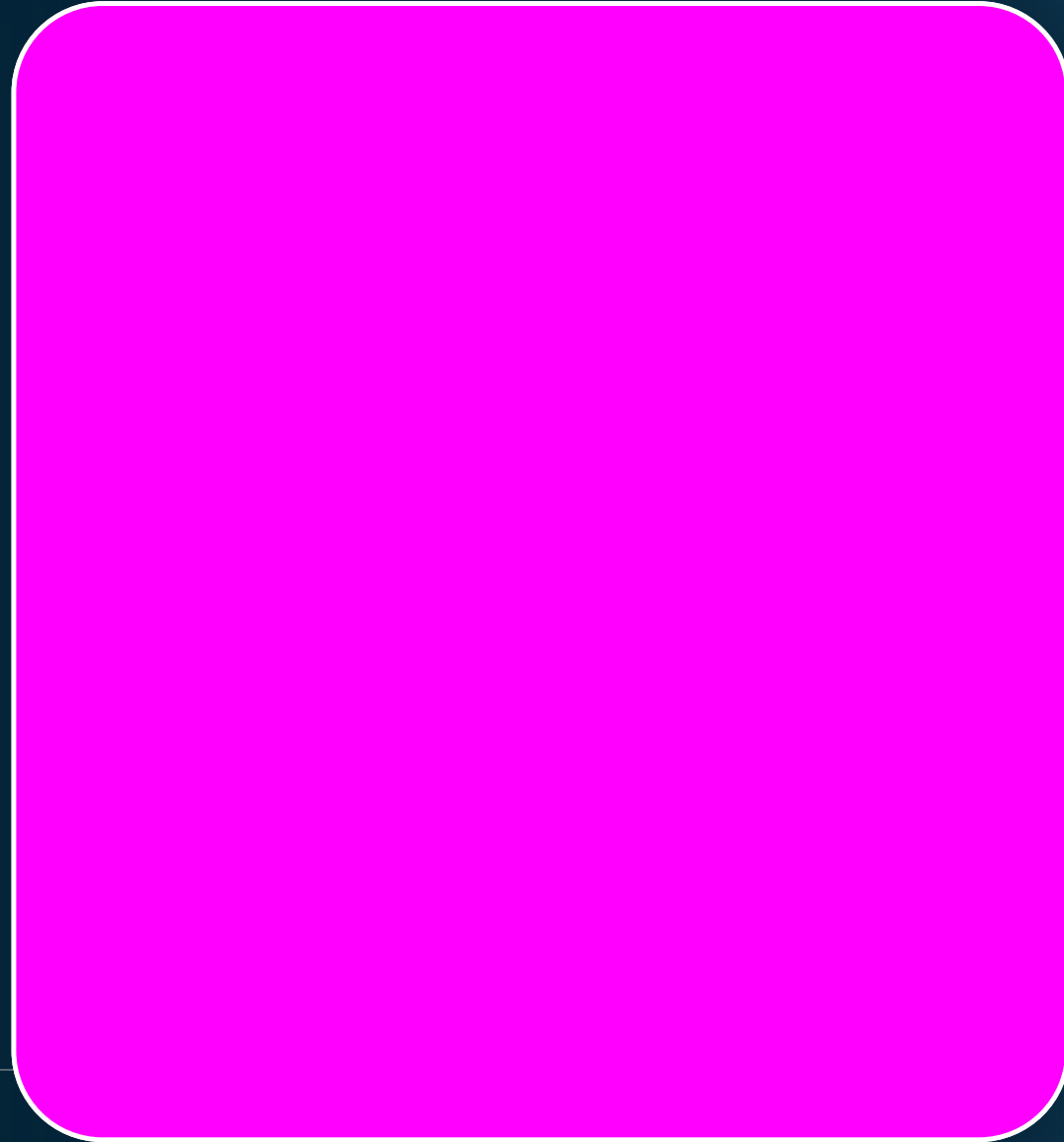
Audit the Data Layer

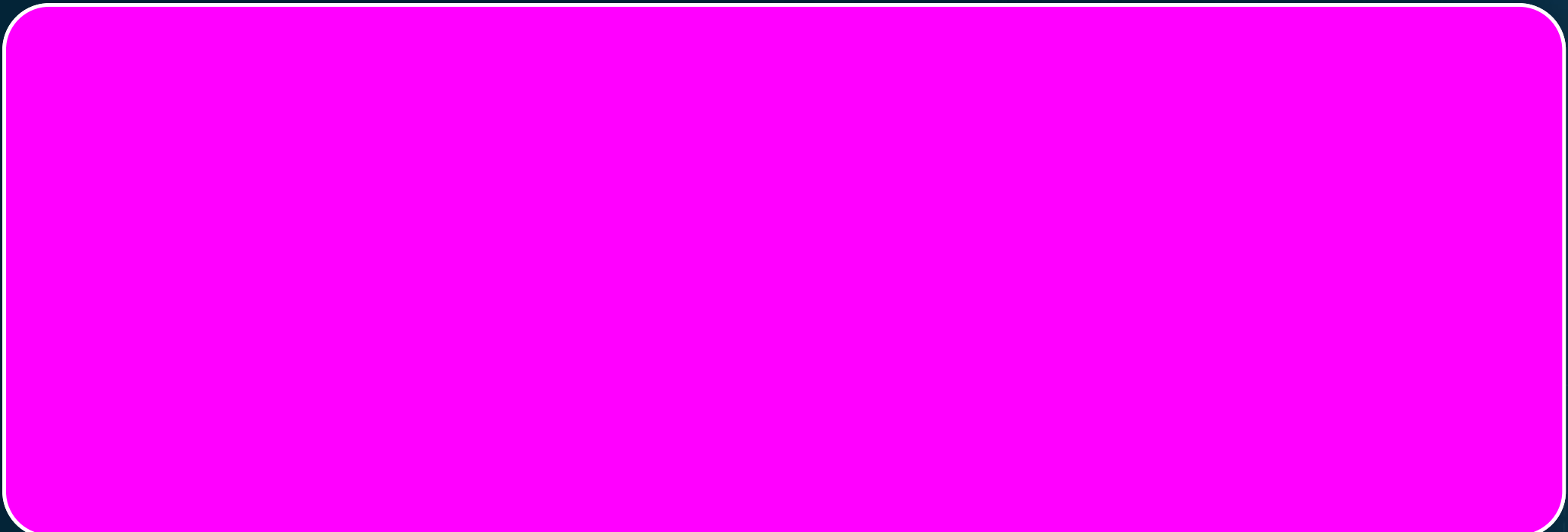
Log who accessed which datasets,
when, what role

Monitor unusual retrieval patterns
(indirect probing)

Correlate with identity: escalation,
geo anomalies

Review permissions each cycle —
stop privilege creep





Demo: Data Access Governance Audit

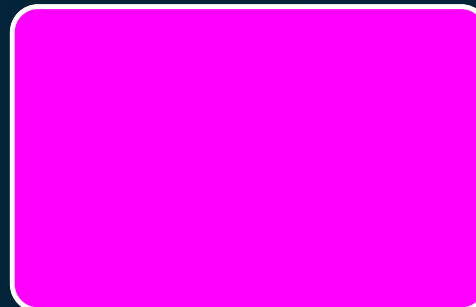
Dataset classification, RBAC, read/write separation, retrieval-layer permissions & access logging with anomaly detection

Exam Tips



TWO LAYERS

Data $\neq$ model — test both



READ vs WRITE

Least privilege on data



RAG SURFACE

Vector leakage is separate

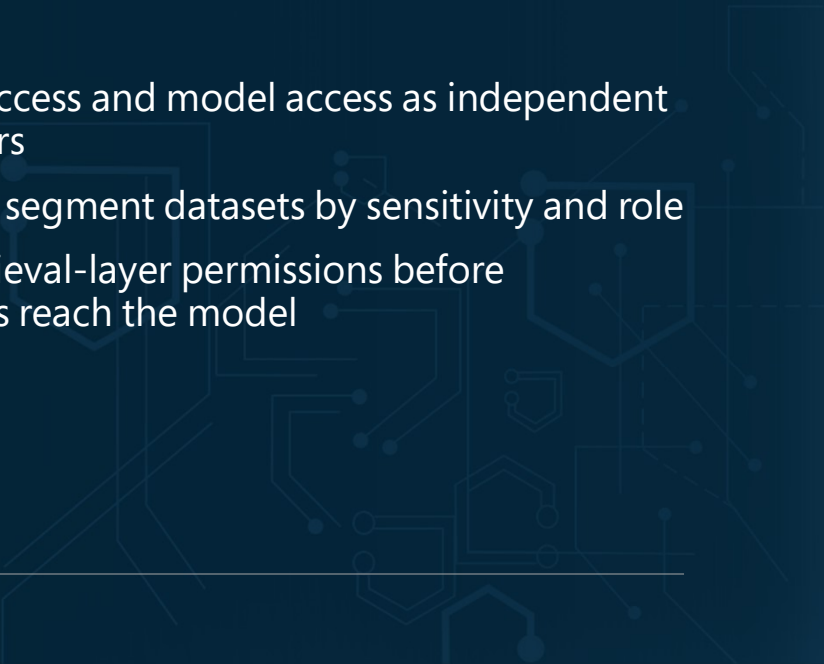
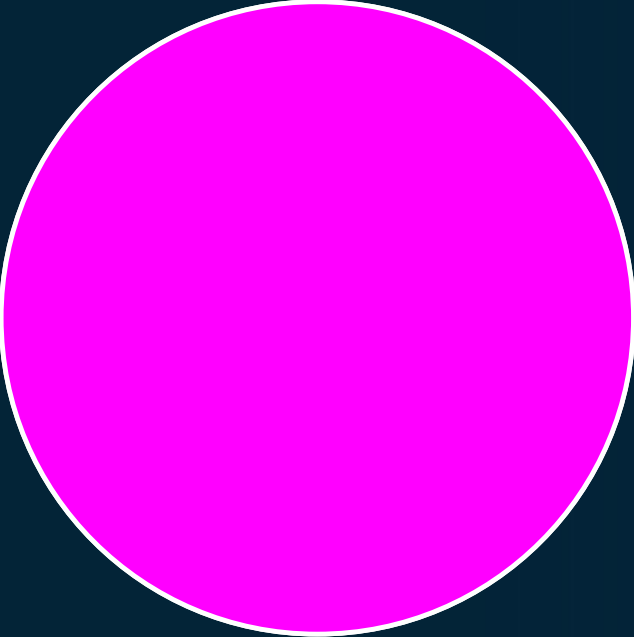


OVERREACH

Retrieval fail, not model fail

Data Access Controls in AI Systems

Chapter 08 · Episode 04 · Domain 2.3 — Data Boundaries Are the Perimeter



Learning Objectives

1. Treat data access and model access as independent control layers
2. Classify and segment datasets by sensitivity and role
3. Enforce retrieval-layer permissions before embeddings reach the model

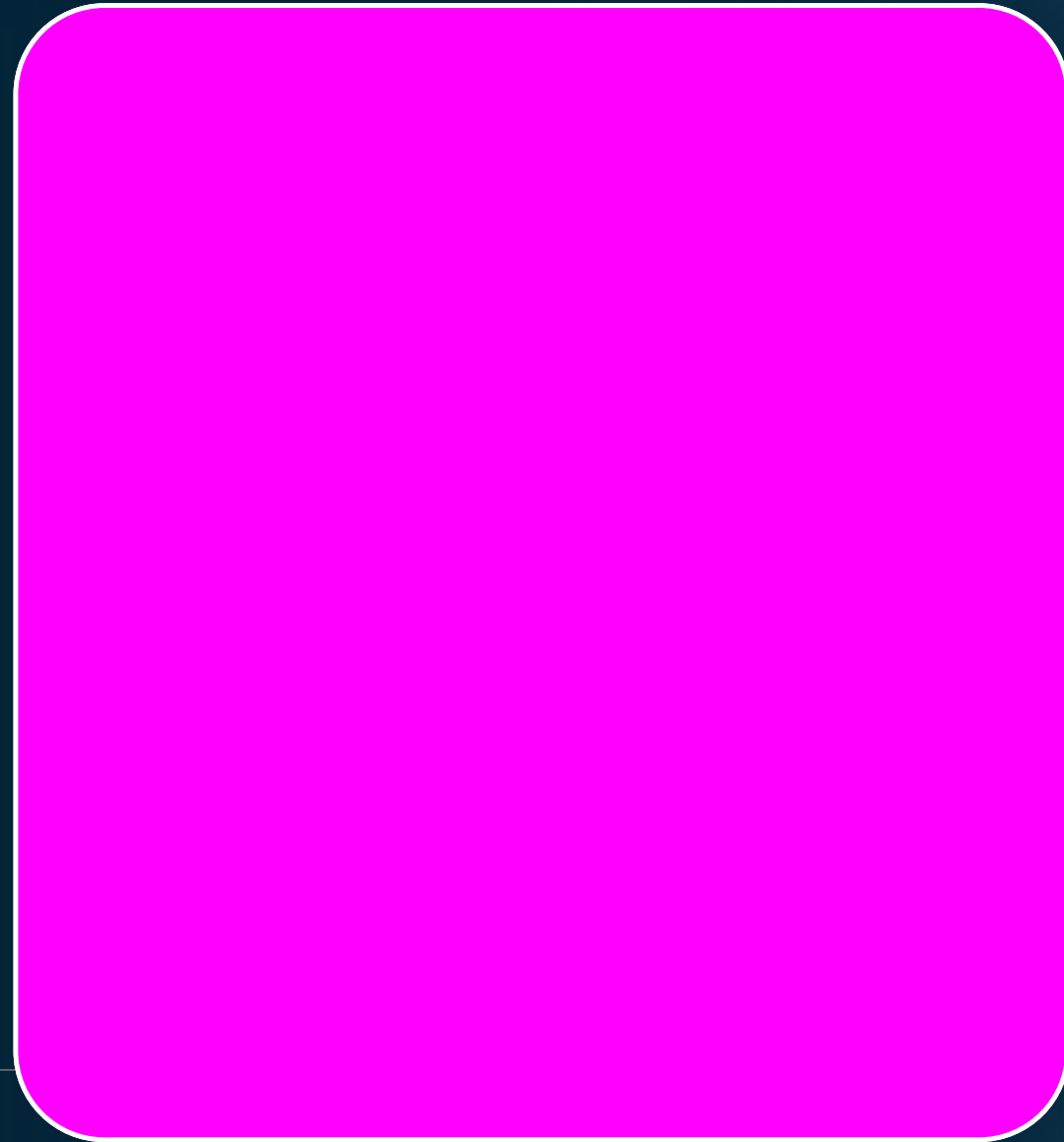
Segment Datasets by Sensitivity

Classify: public → internal →
confidential → regulated

Fraud DB → financial security team only

Endpoint telemetry → SOC analysts only

Watch cross-dataset exposure &
bias amplification



Read vs Write Privilege Separation

READ — MANY

Visibility for evaluation

Monitor excessive inspection

Inspection spikes = recon

WRITE — FEW

Alters dataset contents

Unauthorized write = poisoning

Validated ingestion pipelines only

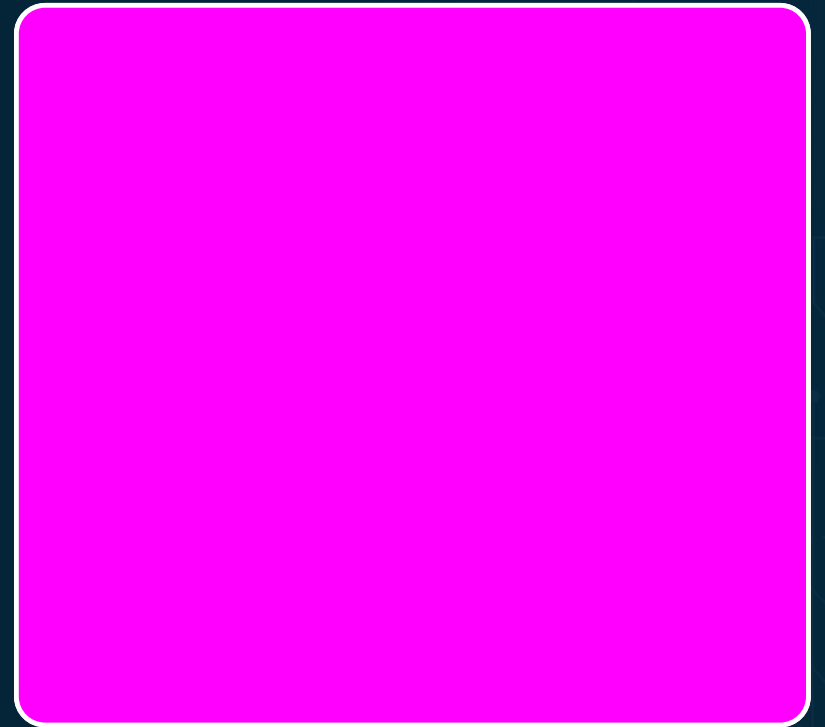
Retrieval Is Its Own Exposure Surface

Role-based retrieval: docs at/below clearance

Filter enforces BEFORE results reach model

Segment sensitive corpora from general KB

Inference-time overreach = retrieval governance fail



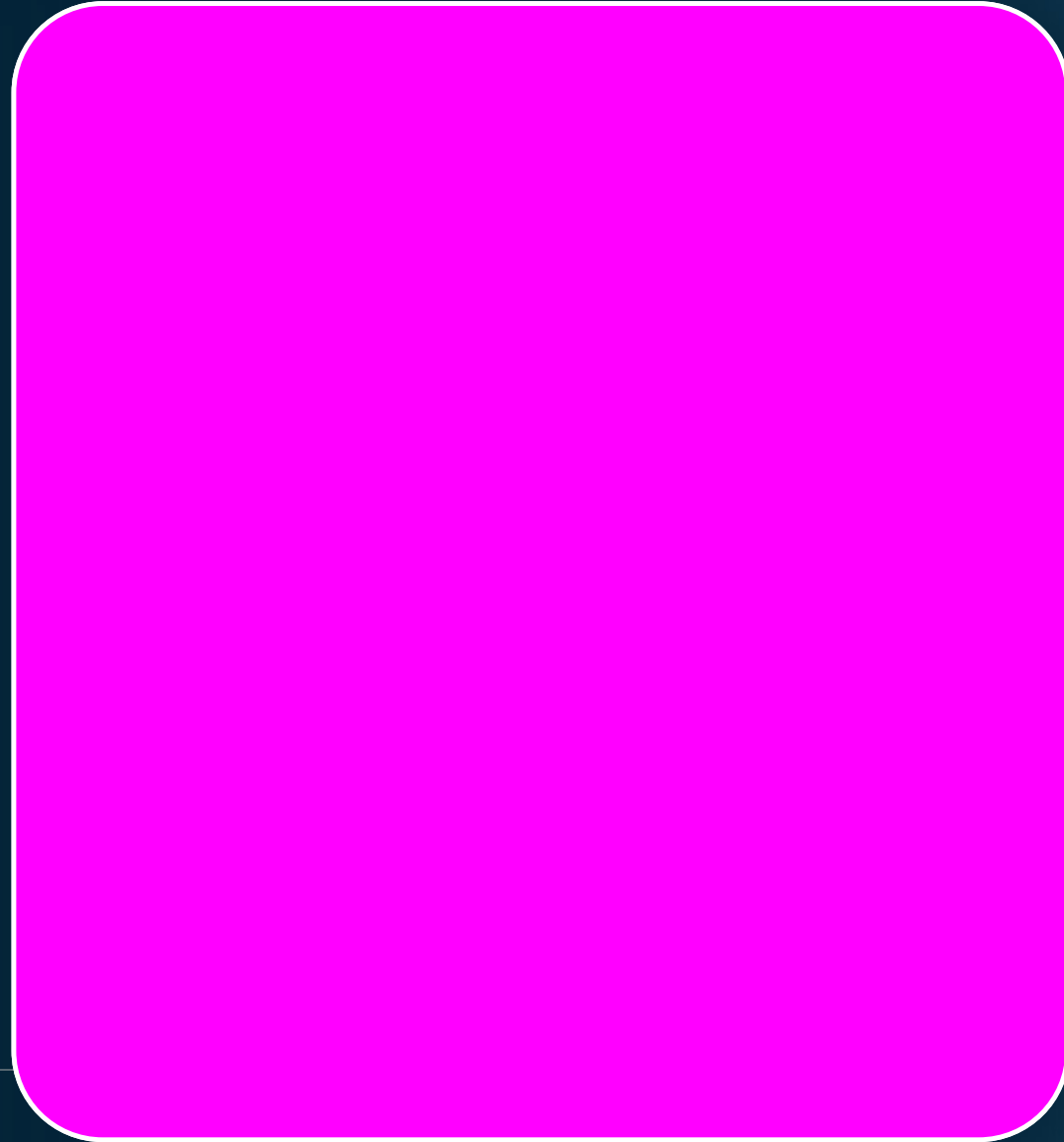
Audit the Data Layer

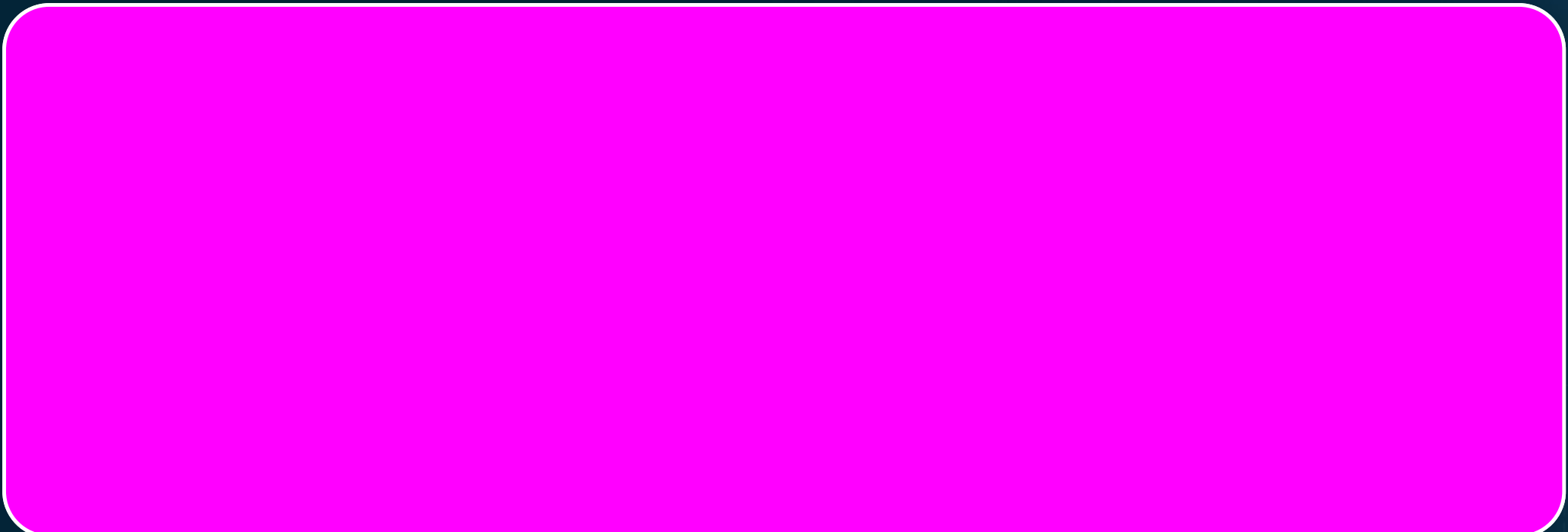
Log who accessed which datasets,
when, what role

Monitor unusual retrieval patterns
(indirect probing)

Correlate with identity: escalation,
geo anomalies

Review permissions each cycle —
stop privilege creep





Demo: Data Access Governance Audit

Dataset classification, RBAC, read/write separation, retrieval-layer permissions & access logging with anomaly detection

Exam Tips



TWO LAYERS

Data $\neq$ model — test both



READ vs WRITE

Least privilege on data



RAG SURFACE

Vector leakage is separate

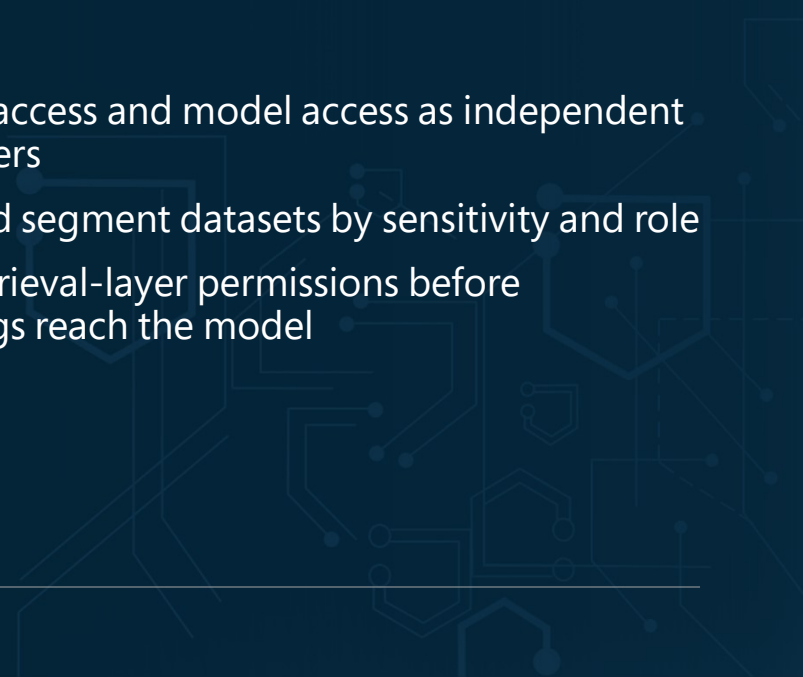
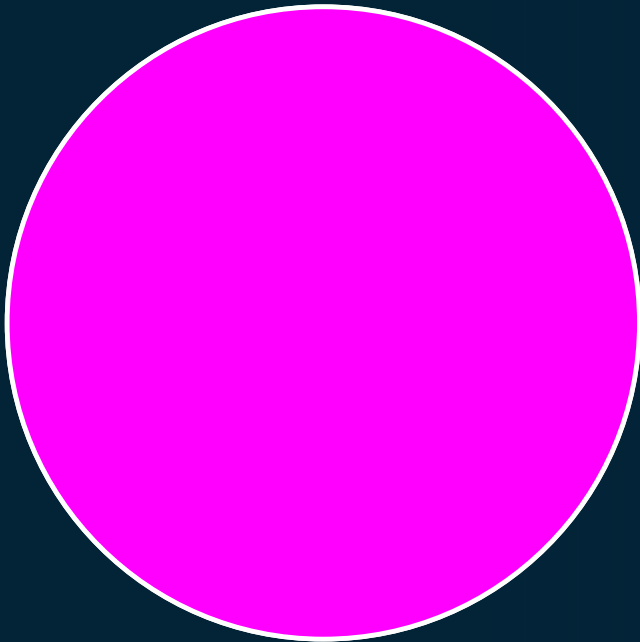


OVERREACH

Retrieval fail, not model fail

Data Access Controls in AI Systems

Chapter 08 · Episode 04 · Domain 2.3 — Data Boundaries Are the Perimeter



Learning Objectives

1. Treat data access and model access as independent control layers
2. Classify and segment datasets by sensitivity and role
3. Enforce retrieval-layer permissions before embeddings reach the model

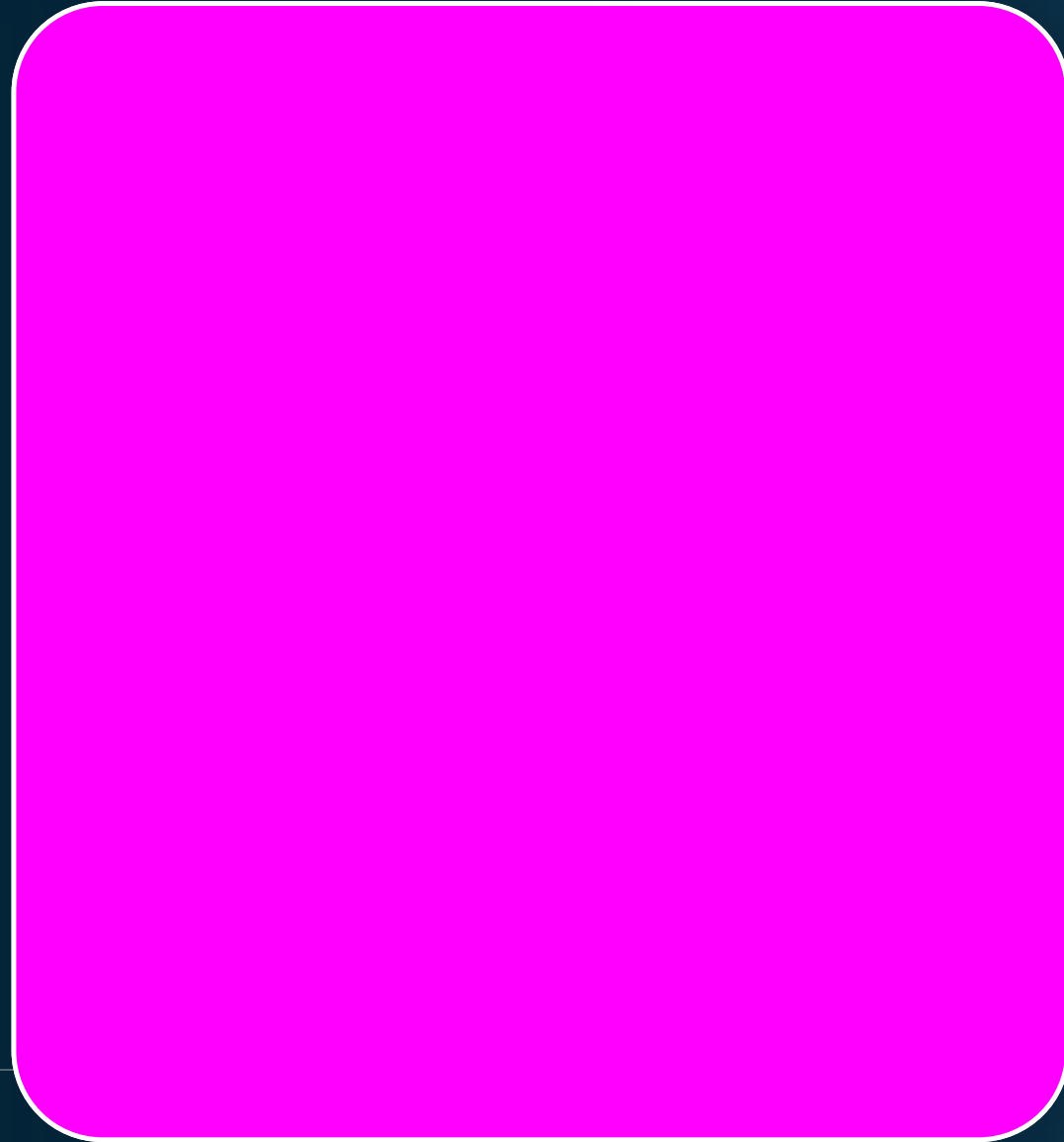
Segment Datasets by Sensitivity

Classify: public → internal →
confidential → regulated

Fraud DB → financial security team only

Endpoint telemetry → SOC analysts only

Watch cross-dataset exposure &
bias amplification



Read vs Write Privilege Separation

READ — MANY

Visibility for evaluation

Monitor excessive inspection

Inspection spikes = recon

WRITE — FEW

Alters dataset contents

Unauthorized write = poisoning

Validated ingestion pipelines only

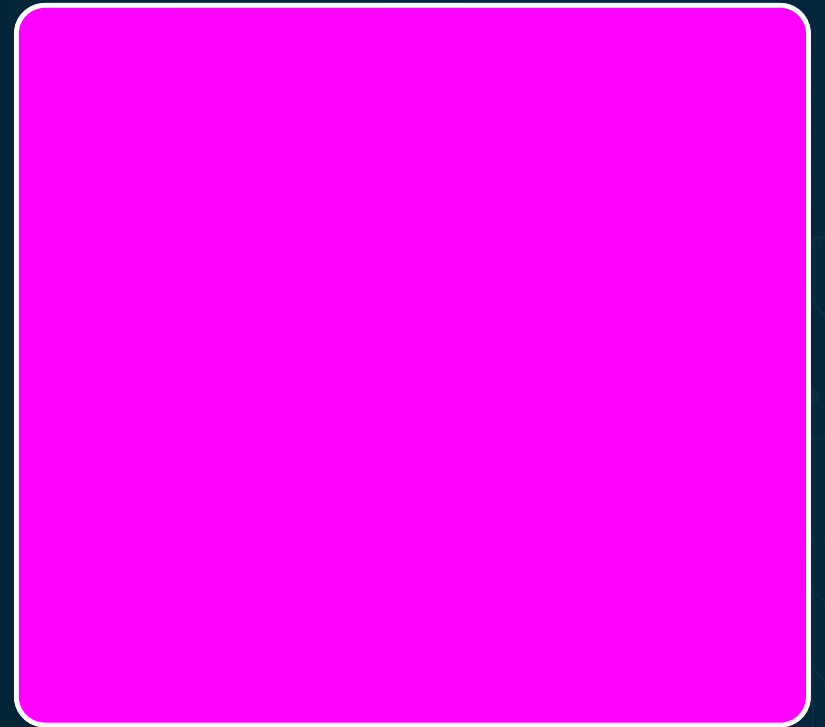
Retrieval Is Its Own Exposure Surface

Role-based retrieval: docs at/below clearance

Filter enforces BEFORE results reach model

Segment sensitive corpora from general KB

Inference-time overreach = retrieval governance fail



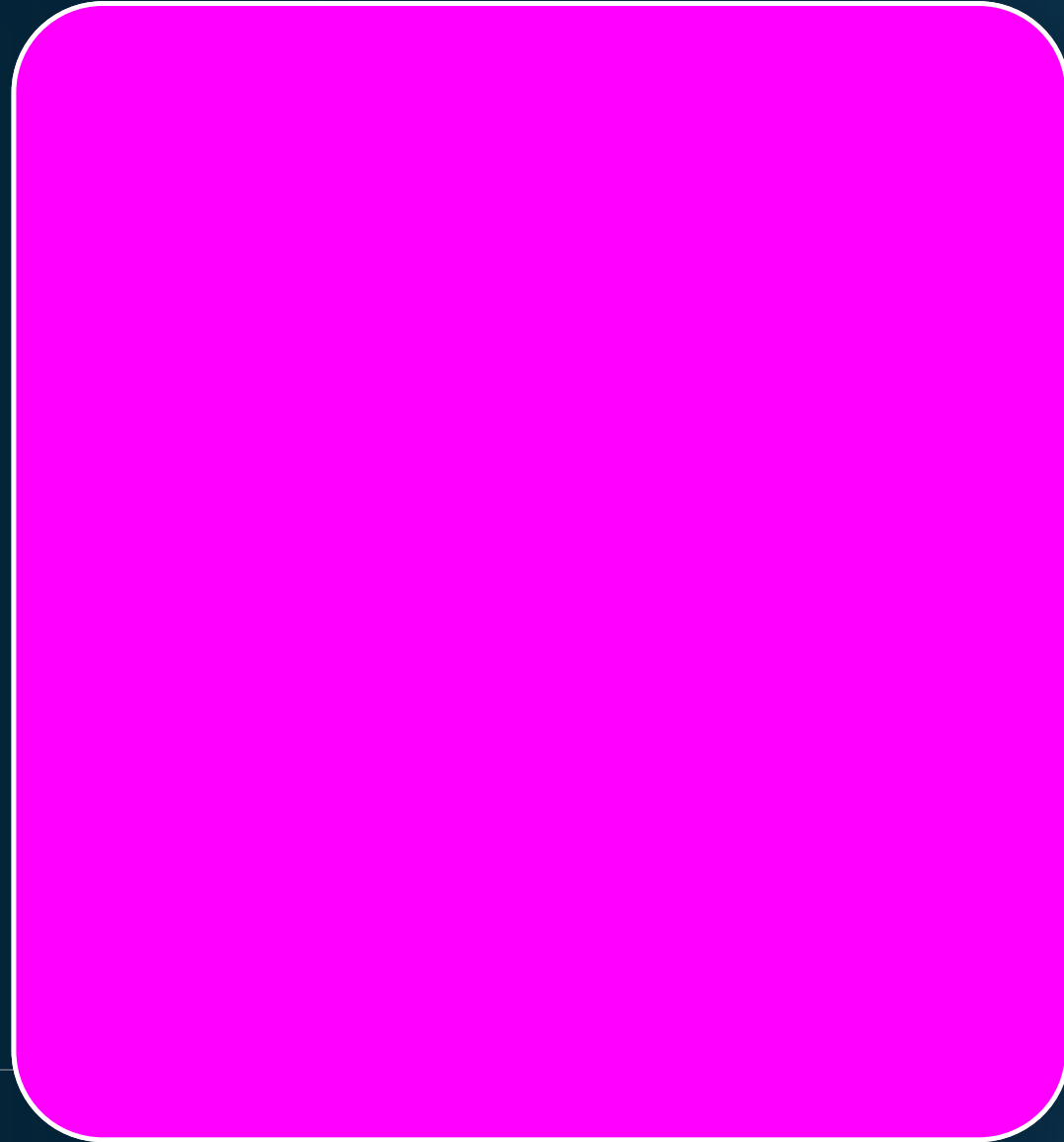
Audit the Data Layer

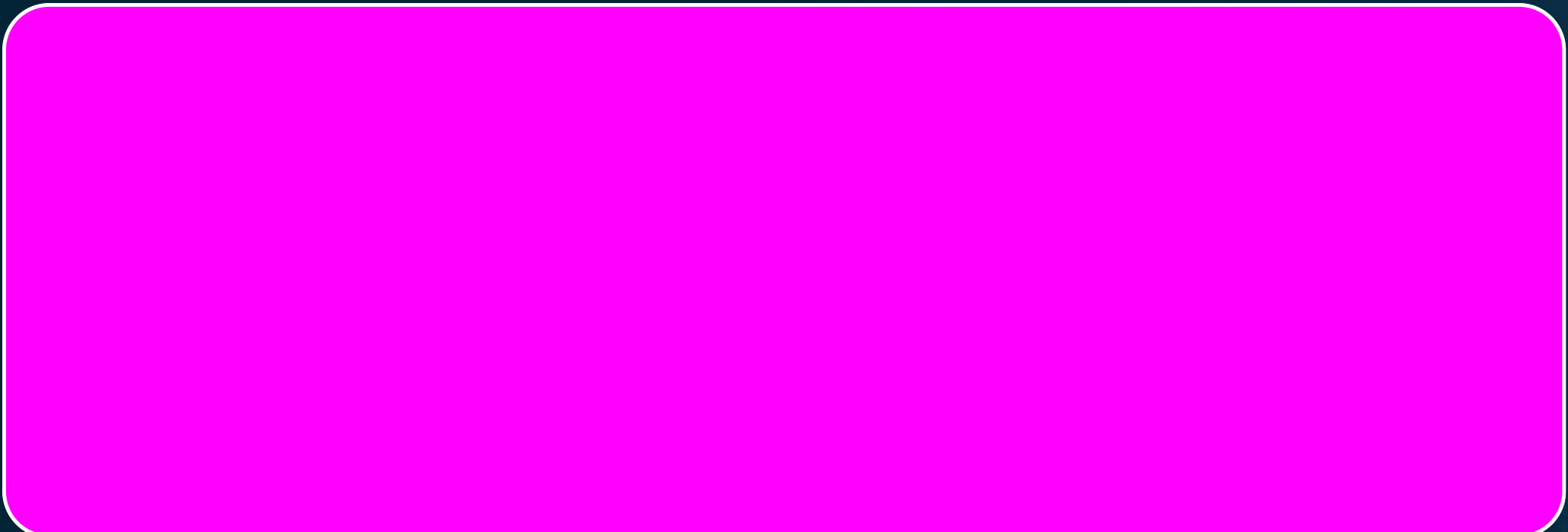
Log who accessed which datasets,
when, what role

Monitor unusual retrieval patterns
(indirect probing)

Correlate with identity: escalation,
geo anomalies

Review permissions each cycle —
stop privilege creep





Demo: Data Access Governance Audit

Dataset classification, RBAC, read/write separation, retrieval-layer permissions & access logging with anomaly detection

Exam Tips



TWO LAYERS

Data $\neq$ model — test both



READ vs WRITE

Least privilege on data



RAG SURFACE

Vector leakage is separate



OVERREACH

Retrieval fail, not model fail